เอกสารประกอบการสอน

วิชา โครงสรางระบบคอมพิวเตอร์

(Computer System Organization)

บทที่ 8 เรื่องการทำงานแบบไปป์ลายน์ (Pipeline)

วัตถุประสงค์เชิงพฤติกรรม (Behavioral Objectives)

1. อธิบายหลักการทำงานของ CPU แบบไม่ใช้ไปป์ไลน์ (Single-cycle) ได้
2. อธิบายหลักการทำงานของ CPU แบบไปป์ไลน์ (Pipeline) ได้
3. อธิบายลำดับขั้นตอนการทำงานของไปป์ไลน์ของสถาปัตยกรรม MIPS ได้
4. อธิบายลำดับขั้นตอนการทำงานของไปป์ไลน์ของสถาปัตยกรรม RISC ได้
5. อธิบายการทำงานไปป์ไลน์แบบเป็นลำดับ (Linear Pipeline) ได้
6. อธิบายการทำงานไปป์ไลน์แบบไม่เป็นลำดับ (Non-linear Pipeline) ได้
7. อธิบายการทำงานโครงสร้างของไปป์ไลน์ (Pipeline Architecture) ได้อย่างถูกต้อง
8. อธิบายการการเพิ่มประสิทธิภาพของไปป์ไลน์ (Pipeline Optimization) ได้อย่างถูกต้อง
9. อธิบายปัญหาในการทำงานแบบไปป์ไลน์ (Pipeline Hazards) แบบข้อมูล (Data Hazard) ได้
10. อธิบายปัญหาในการทำงานแบบไปป์ไลน์ (Pipeline Hazards) แบบโครงสร้าง (Structural Hazard) ได้
11. อธิบายปัญหาในการทำงานแบบไปป์ไลน์ (Pipeline Hazards) แบบควบคุม (Control Hazard) ได้
12. อธิบายเทคนิคการแก้ปัญหา Hazard แบบการหยุดรอ (Stalling) หรือ Bubble ได้
13. อธิบายเทคนิคการแก้ปัญหา Hazard แบบการส่งต่อข้อมูล (Forwarding) หรือ Bypassing ได้
14. อธิบายเทคนิคการแก้ปัญหา Hazard แบบการทำนายการทำงาน (Branch Prediction) ได้
15. อธิบายเทคนิคการแก้ปัญหา Hazard แบบการจัดลำดับคำสั่งใหม่ (Instruction Reordering) ได้
16. อธิบายการเปรียบเทียบประสิทธิภาพการทำงานแบบไปป์ไลน์และการทำงานไม่ใช้ไปป์ไลน์ (Compare the performance of Pipelined vs. Non-pipelined operation) ได้

1.การนำเข้าสู่บทเรียน Introduction to Pipeline

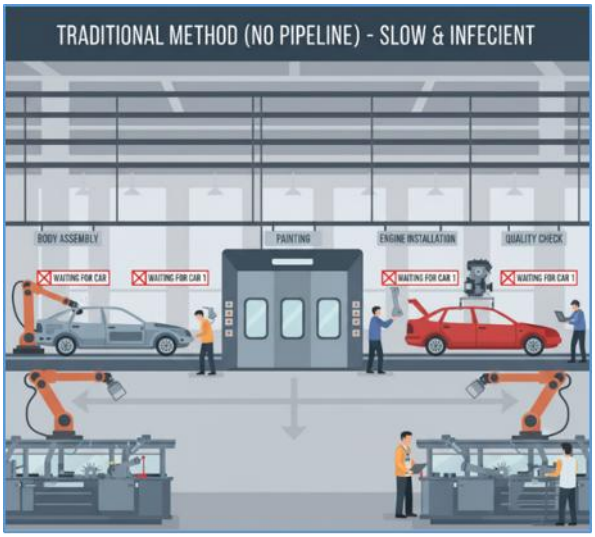
การทำงานแบบไปป์ไลน์ (Pipeline) คือ เทคนิคการเพิ่มประสิทธิภาพของหน่วยประมวลผลกลาง (CPU) ด้วยการแบ่งขั้นตอนการประมวลผลคำสั่งที่เคยทำแบบต่อเนื่อง (Sequential) ให้สามารถทำไปพร้อมๆ กันได้ในแต่ละขั้นตอนโดย**หลักการสำคัญดังนี้**

การแบ่งงานย่อย: งานหนึ่งถูกแบ่งออกเป็นหลายขั้นตอนย่อย ๆ เช่น ขั้นตอนที่ 1, ขั้นตอนที่ 2, ขั้นตอนที่ 3

การทำพร้อมกัน: เมื่อขั้นตอนแรกของงานที่ 1 เสร็จแล้ว งานที่ 2 ก็เข้ามาทำในขั้นตอนแรกทันที ขณะที่งานที่ 1 กำลังเข้าสู่ขั้นตอนที่ 2

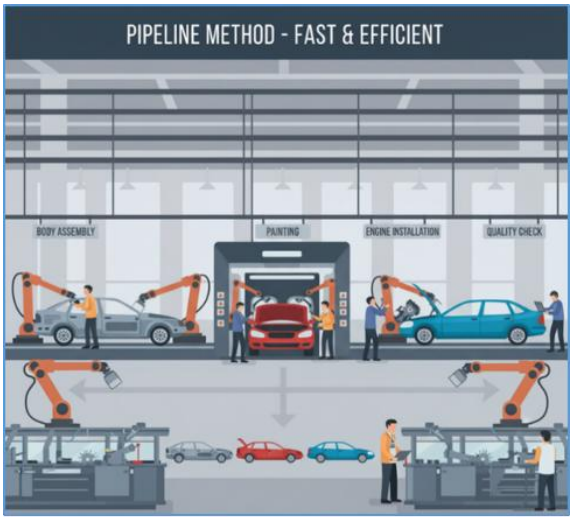
การทำต่อเนื่อง: แต่ละขั้นตอนจะทำงานอย่างต่อเนื่อง ส่งผลลัพธ์ไปยังขั้นตอนถัดไป ทำให้ไม่เกิดการรอคอย

<u>ตัวอย่างการทำงานของแบบไปป์ไลน์ (Pipeline)</u> "โรงงานผลิตรถยนต์"



รูปที่ 8 - 1 ภาพจำลองกระบวนการทำงานของสถาปัตยกรรม CPU แบบไม่มีระบบไปป์ไลน์

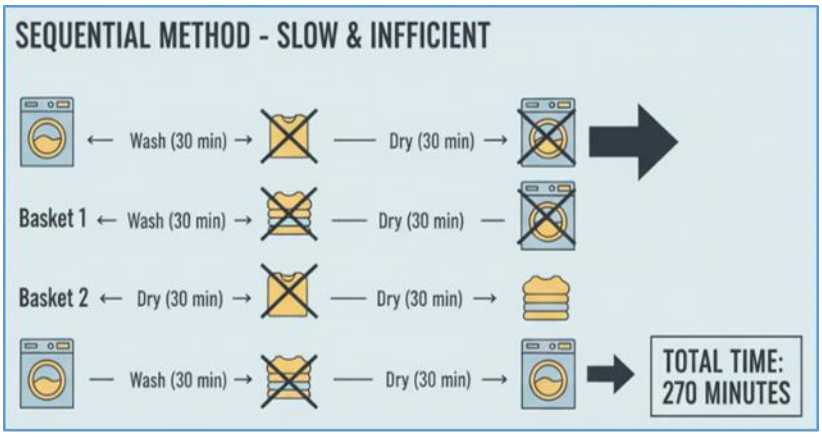
- **แบบดั้งเดิม (ไม่ใช้ไปป์ไลน์)**: รถแต่ละคันต้องผ่านกระบวนการ **ประกอบตัวถัง** -> **ลงสี** -> **ติดตั้งเครื่องยนต์** -> **ตรวจสอบคุณภาพ** ทีละขั้นตอนจนเสร็จสมบูรณ์ รถคันที่ 2 จะเริ่มผลิตได้ก็ต่อเมื่อรถคันแรกเสร็จสิ้นกระบวนการทั้งหมดแล้วเท่านั้น ทำให้ใช้เวลานานและไม่คุ้มค่า
- **แบบไปป์ไลน์**: โรงงานจะแบ่งออกเป็นหลายๆ สถานี (หรือ "ขั้น") โดยที่แต่ละสถานีจะทำงานที่แตกต่างกัน เช่น สถานีที่ 1 ทำการประกอบตัวถัง, สถานีที่ 2 ทำการลงสี, สถานีที่ 3 ติดตั้งเครื่องยนต์ และสถานีที่ 4 ตรวจสอบคุณภาพ
  - เมื่อรถคันที่ 1 ประกอบตัวถังเสร็จ ก็จะส่งต่อไปยังสถานีลงสีทันที
  - ขณะที่รถคันที่ 1 กำลังลงสี **รถคันที่ 2 ก็สามารถเริ่มกระบวนการประกอบตัวถังที่สถานีแรกได้ทันที**
  - เมื่อเวลาผ่านไปเรื่อยๆ รถทุกคันจะเคลื่อนที่ไปตามสถานีต่างๆ พร้อมกัน ทำให้ในแต่ละช่วงเวลาจะมีรถหลายคันอยู่ในขั้นตอนการผลิตที่แตกต่างกัน
  - ผลลัพธ์คือ: **ใช้เวลาโดยรวมลดลง** และ **ได้จำนวนรถที่ผลิตเสร็จต่อหน่วยเวลาเพิ่มขึ้น** (ซึ่งเรียกว่า Throughput)



รูปที่ 8 – 2 ภาพจำลองกระบวนการทำงานของสถาปัตยกรรม CPU แบบมีระบบไปป์ไลน์

ตัวอย่าง: การซักผ้าแบบไปป์ไลน์ (เพื่อความเข้าใจง่าย ลองเปรียบเทียบการซักผ้า 3 ถัง ดังนี้)

1. แบบไม่ใช้ไปป์ไลน์ (Sequential) คือ



รูปที่ 8 – 3 ภาพจำลองการประมวลผลแบบตามลำดับ (Sequential Method) ด้วยกิจกรรมการซักและอบผ้า

ขั้นตอนเหมือนกับการทำทีละอย่างจนเสร็จ:

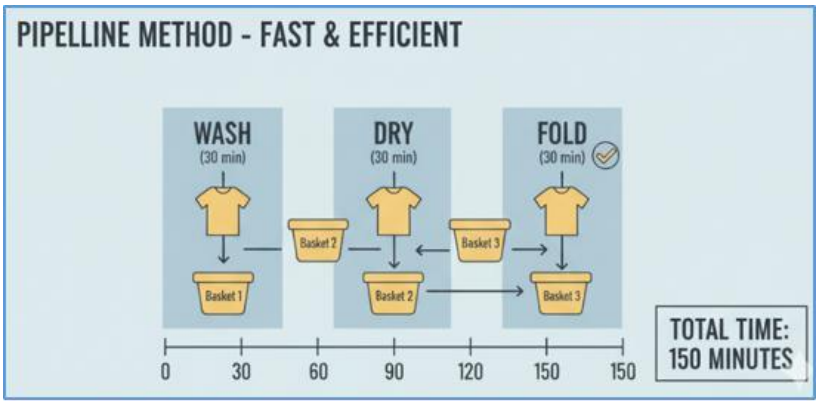
ถังที่ 1: ซัก (30 นาที) -> ตาก (30 นาที) -> พับ (30 นาที) = **รวม 90 นาที**

ถังที่ 2: เริ่มเมื่อถังแรกเสร็จ = **รวม 90 + 90 = 180 นาที**

ถังที่ 3: เริ่มเมื่อถังที่สองเสร็จ = **รวม 180 + 90 = 270 นาที**

สรุปจะเห็นว่าใช้เวลานานมาก เพราะต้องรอให้งานแต่ละถังเสร็จสมบูรณ์ก่อนถึงจะเริ่มถัดไปได้

2. แบบใช้ไปป์ไลน์ (Pipeline) เหมือนกับการทำแต่ละขั้นตอนพร้อม ๆ กัน:



รูปที่ 8 – 4 ภาพจำลองการประมวลผลแบบไปป์ไลน์ (Pipeline Method) ด้วยกิจกรรมการซัก อบ และพับผ้า

- **ขั้นตอนที่ 1 (ซัก):** ใช้เวลา 30 นาที
- **ขั้นตอนที่ 2 (ตาก):** ใช้เวลา 30 นาที
- **ขั้นตอนที่ 3 (พับ):** ใช้เวลา 30 นาที

มาดูการทำงานจริง:

- **เวลา 0-30 นาที:** เริ่มซักถังที่ 1
- **เวลา 30-60 นาที:** ถังที่ 1 ย้ายไปตาก, เริ่มซักถังที่ 2
- **เวลา 60-90 นาที:** ถังที่ 1 ย้ายไปพับ, ถังที่ 2 ย้ายไปตาก, เริ่มซักถังที่ 3
- **เวลา 90 นาที:** ถังที่ 1 พับเสร็จ
- **เวลา 120 นาที:** ถังที่ 2 พับเสร็จ
- **เวลา 150 นาที:** ถังที่ 3 พับเสร็จ

สรุป: งานทั้งหมด 3 ถังเสร็จในเวลาเพียง **150 นาที** ซึ่งเร็วกว่าแบบแรกเกือบครึ่ง นี่คือหลักการของไปป์ไลน์: **เพิ่มประสิทธิภาพโดยรวม (Throughput)** แม้ว่าแต่ละงาน (Latency) จะใช้เวลาเท่าเดิม

2. การเปรียบเทียบในเชิง CPU การทำงานแบบไปป์ไลน์ (Pipeline)

ในหน่วยประมวลผล (CPU) การทำงานแบบไปป์ไลน์ก็ใช้หลักการเดียวกัน โดยแบ่งการประมวลผลคำสั่งออกเป็นขั้นๆ (Pipeline Stages) เช่น:

- **ขั้นที่ 1: Fetch (IF)**: ดึงคำสั่งจากหน่วยความจำ
- **ขั้นที่ 2: Decode (ID)**: ถอดรหัสคำสั่งและเตรียมข้อมูล
- **ขั้นที่ 3: Execute (EX)**: ประมวลผลคำสั่ง
- **ขั้นที่ 4: Memory Access (MEM)**: เข้าถึงหน่วยความจำ (ถ้าจำเป็น)
- **ขั้นที่ 5: Write Back (WB)**: เขียนผลลัพธ์กลับไปยังรีจิสเตอร์

2.1 แบบไม่ใช้ไปป์ไลน์ (Sequential)

คำสั่ง	Clock Cycle 1	Clock Cycle 2	Clock Cycle 3	Clock Cycle 4	Clock Cycle 5
คำสั่งที่ 1	IF	ID	EX	MEM	WB
คำสั่งที่ 2		IF	ID	EX	MEM
คำสั่งที่ 3			IF	ID	EX

ตารางที่ 8-1 ตารางแสดงลำดับขั้นตอนการประมวลผลคำสั่งแบบไม่ใช้ไปป์ไลน์ (Sequential)

2.2 แบบใช้ไปป์ไลน์ (Pipeline)

คำสั่ง	Clock Cycle 1	Clock Cycle 2	Clock Cycle 3	Clock Cycle 4	Clock Cycle 5	Clock Cycle 6	Clock Cycle 7
คำสั่งที่ 1	IF	ID	EX	MEM	WB		
คำสั่งที่ 2		IF	ID	EX	MEM	WB	
คำสั่งที่ 3			IF	ID	EX	MEM	WB
คำสั่งที่ 4				IF	ID	EX	MEM

ตารางที่ 8-2 ตารางแสดงลำดับขั้นตอนการประมวลผลคำสั่งแบบใช้ไปป์ไลน์ (Pipeline)

- ใน Clock Cycle ที่ 3 จะมี **3 คำสั่งที่ทำงานพร้อมกัน** ในคนละขั้น (คำสั่งที่ 1 อยู่ในขั้น EX, คำสั่งที่ 2 อยู่ในขั้น ID, คำสั่งที่ 3 อยู่ในขั้น IF)
- ผลลัพธ์คือ เมื่อไปป์ไลน์ทำงานจนเต็ม (Full) จะสามารถประมวลผลคำสั่งใหม่ได้ **เกือบทุก Clock Cycle** ทำให้ประสิทธิภาพเพิ่มขึ้นอย่างมาก

***ข้อสรุป**: จากภาพจะเห็นว่าใน Clock Cycle ที่ 5 คำสั่งทุกคำสั่งอยู่ในแต่ละขั้น ทำให้ระบบทำงานได้อย่างเต็มที่ การทำงานแบบไปป์ไลน์ช่วย **ลดเวลาเฉลี่ยในการประมวลผลแต่ละคำสั่ง** (Instruction Latency) และ **เพิ่มจำนวนคำสั่งที่ประมวลผลเสร็จต่อหน่วยเวลา** (Throughput) ได้อย่างมาก แม้ว่าแต่ละคำสั่งจะใช้เวลาในการทำงานเท่าเดิม แต่การทำแบบขนานจะทำให้ภาพรวมของระบบเร็วขึ้น*

3. สรุปการทำงานแบบไปป์ไลน์ (Pipeline)

หลักการพื้นฐานของไปป์ไลน์: หลักการของไปป์ไลน์คือการแบ่ง **วงรอบคำสั่ง (Instruction Cycle)** ที่ปกติจะทำงานแบบเรียงลำดับ (sequential) ออกเป็นขั้นตอนย่อย ๆ แล้วให้แต่ละขั้นตอนทำงานพร้อมกัน ซึ่งจะช่วยเพิ่มประสิทธิภาพและความเร็วในการประมวลผลโดยรวมได้อย่างมาก

3.1 หลักการทำงานของ CPU: Single-cycle vs. Pipeline

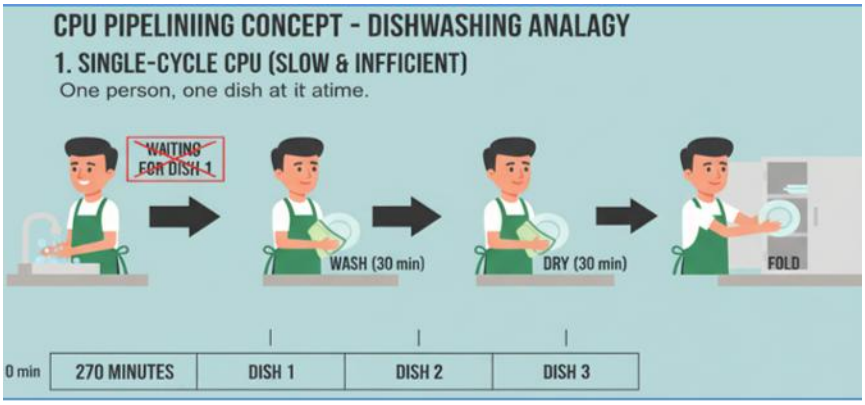
การทำงานของหน่วยประมวลผลกลาง (CPU) สามารถแบ่งออกเป็น 2 แบบหลักตามวิธีการประมวลผลคำสั่ง ได้แก่ **แบบไม่ใช้ไปป์ไลน์ (Single-cycle)** และ **แบบใช้ไปป์ไลน์ (Pipeline)** ซึ่งทั้งสองแบบมีหลักการทำงานที่แตกต่างกันอย่างสิ้นเชิง ส่งผลต่อความเร็วและประสิทธิภาพโดยรวมของคอมพิวเตอร์

3.1.1 CPU แบบไม่ใช้ไปป์ไลน์ (Single-cycle CPU)

หลักการทำงาน: CPU แบบ Single-cycle จะประมวลผล **หนึ่งคำสั่งให้เสร็จสมบูรณ์ภายในหนึ่งรอบสัญญาณนาฬิกา (One Clock Cycle)** หมายความว่า ทุกขั้นตอนของวัฏจักรคำสั่ง (Fetch, Decode, Execute, Memory, Write Back) จะเกิดขึ้นและจบลงภายในรอบสัญญาณนาฬิกาเดียวกัน ก่อนที่จะเริ่มต้นประมวลผลคำสั่งถัดไป

ข้อดี: ออกแบบง่าย: โครงสร้างไม่ซับซ้อน ทำให้เข้าใจง่ายและสร้างได้ง่าย

ไม่มี Hazard: เนื่องจากแต่ละคำสั่งทำงานเสร็จก่อนที่จะเริ่มคำสั่งใหม่ จึงไม่มีปัญหาเรื่อง Data, Structural หรือ Control Hazard



รูปที่ 8 – 5 ภาพจำลองการประมวลผลของ CPU แบบซิงเกิลไซเคิล (Single-Cycle) ด้วยกิจกรรมการล้างจาน

ข้อเสีย: ความเร็วต่ำ: ความเร็วของรอบสัญญาณนาฬิกาจะถูกกำหนดโดยขั้นตอนที่ช้าที่สุด (เช่น การเข้าถึงหน่วยความจำ) ทำให้ CPU ต้องรอให้ขั้นตอนที่ช้าที่สุดเสร็จก่อนถึงจะเริ่มคำสั่งต่อไปได้

ประสิทธิภาพต่ำ: สามารถประมวลผลได้เพียง 1 คำสั่งต่อ 1 รอบสัญญาณนาฬิกาเท่านั้น

ตัวอย่างเปรียบเทียบ: ลองนึกภาพการทำงานของ **คนล้างจาน 1 คน** ที่ต้องทำทุกขั้นตอน (ล้าง, เช็ด, เก็บ) ให้เสร็จในจานใบแรก ก่อนที่จะเริ่มล้างจานใบที่สอง

- **จานใบที่ 1:** ล้าง ⟶ เช็ด ⟶ เก็บ **(เสร็จ)**
- **จานใบที่ 2:** เริ่มต้นทำใหม่ตั้งแต่ล้าง

3.1.2 CPU แบบใช้ไปป์ไลน์ (Pipeline CPU)

หลักการทำงาน: CPU แบบไปป์ไลน์จะแบ่งวัฏจักรคำสั่งออกเป็น **หลายขั้นตอน (Stages)** และให้แต่ละขั้นตอนทำงานพร้อมกัน โดยแต่ละสเตจจะประมวลผลคนละคำสั่งกันไปตามลำดับ คล้ายกับการทำงานบน **สายพานการผลิต** ที่แต่ละคนในสายพานจะทำหน้าที่ของตัวเอง และส่งงานต่อให้คนถัดไปทันทีเมื่อเสร็จ

ข้อดี: **ความเร็วสูง:** แม้ว่าแต่ละคำสั่งจะใช้เวลาประมวลผลทั้งหมดเท่ากับแบบ Single-cycle แต่ CPU สามารถเริ่มต้นประมวลผลคำสั่งใหม่ได้ทุกรอบสัญญาณนาฬิกา

ประสิทธิภาพสูง: สามารถประมวลผลได้หลายคำสั่งในเวลาเดียวกัน ทำให้มี **Throughput** (ปริมาณงานที่ทำเสร็จในหนึ่งหน่วยเวลา) สูงกว่ามาก

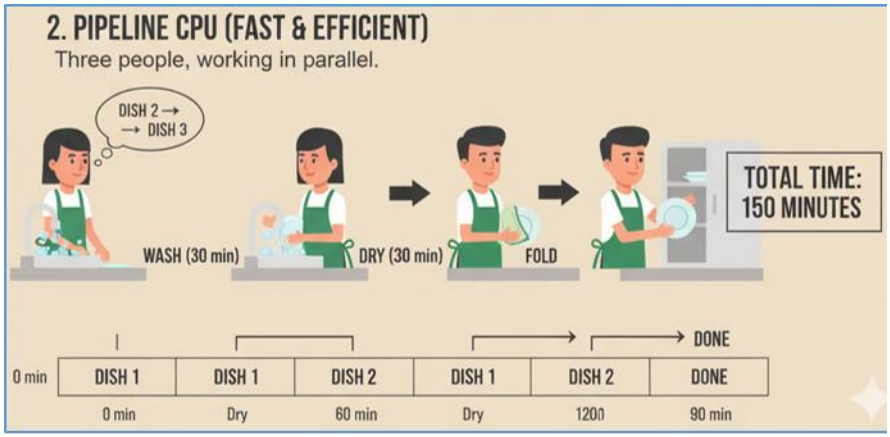
ข้อเสีย: โครงสร้างซับซ้อน: ต้องมีหน่วยเก็บข้อมูลชั่วคราว (Buffer/Latch) ระหว่างแต่ละสเตจ และวงจรควบคุมที่ซับซ้อน

เกิด Hazard: มีโอกาสเกิดปัญหา Pipeline Hazards ต่างๆ ที่ต้องมีกลไกในการจัดการ เช่น Data Hazard, Structural Hazard, และ Control Hazard

ตัวอย่างเปรียบเทียบ: เปรียบเสมือนการทำงานของ **คนล้างจาน 3 คน** ที่แบ่งหน้าที่กัน

- **คน 1:** ล้างจาน
- **คน 2:** เช็ดจาน
- **คน 3:** เก็บจานเข้าตู้

เมื่อคน 1 ล้างจานใบที่ 1 เสร็จ ก็จะส่งต่อไปให้คน 2 ทันที และคน 1 ก็จะเริ่มล้างจานใบที่ 2 ต่อไปเรื่อยๆ ทำให้งานเดินหน้าอย่างต่อเนื่องและเสร็จเร็วขึ้นกว่าแบบแรก



รูปที่ 8 – 6 ภาพจำลองการประมวลผลของ CPU แบบไปป์ไลน์ (Pipeline) ด้วยกิจกรรมการล้างจานแบบคู่ขนาน

คุณสมบัติ	Single-cycle CPU	Pipeline CPU
จำนวนคำสั่งต่อรอบ	1 คำสั่ง	1 คำสั่ง (ในทางทฤษฎี)
จำนวนคำสั่งที่ทำงานพร้อมกัน	1 คำสั่ง	มากกว่า 1 คำสั่ง
ประสิทธิภาพโดยรวม	ต่ำ	สูง
ความซับซ้อน	ต่ำ	สูง
ปัญหาที่เกิดขึ้น	ไม่มี Hazard	มี Hazard

ตารางที่ 8-3 เปรียบเทียบคุณสมบัติข้อดี-ข้อเสียระหว่าง Single-cycle CPU และ Pipeline CPU

4.ลำดับขั้นตอนการทำงานของไปป์ไลน์ในสถาปัตยกรรม MIPS/RISC ทั่วไป

การทำงานของไปป์ไลน์ในหน่วยประมวลผลที่มีสถาปัตยกรรมแบบ **MIPS (Microprocessor without Interlocked Pipeline Stages)** หรือ **RISC (Reduced Instruction Set Computing)** โดยทั่วไปจะแบ่งออกเป็น **5 ขั้นตอนหลัก (5-stage pipeline)** เพื่อให้การประมวลผลคำสั่งมีประสิทธิภาพสูงสุด แต่ละขั้นตอนจะทำงานโดยอิสระและส่งผลลัพธ์ต่อไปยังขั้นตอนถัดไปอย่างต่อเนื่อง

ขั้นตอนหลักของไปป์ไลน์ 5 สเตจ

4.1 สเตจ IF: Instruction Fetch (ดึงคำสั่ง) 📥

- **หน้าที่:** ดึงคำสั่งจากหน่วยความจำ (Instruction Memory) โดยใช้ค่าจาก Program Counter (PC)
- **การทำงาน:** CPU จะนำค่า PC ไปชี้ตำแหน่งคำสั่งในหน่วยความจำ แล้วดึงคำสั่งนั้นมาเก็บไว้ในบัฟเฟอร์ เพื่อส่งต่อให้สเตจถัดไป พร้อมกับเพิ่มค่า PC เพื่อเตรียมดึงคำสั่งต่อไปในรอบสัญญาณนาฬิกาถัดไป

4.2 สเตจ ID: Instruction Decode (ถอดรหัสคำสั่ง) □

- **หน้าที่:** ถอดรหัสคำสั่งที่ได้รับมาจากสเตจ IF เพื่อระบุว่าคำสั่งนั้นคืออะไร (เช่น บวก, ลบ, โหลดข้อมูล) และอ่านค่าข้อมูลจากรีจิสเตอร์ (Register File) ที่คำสั่งนั้นระบุไว้
- **การทำงาน:** คำสั่งจะถูกแยกส่วนเพื่อระบุประเภท (Opcode) และรีจิสเตอร์ที่เกี่ยวข้อง (Source/Destination Registers) จากนั้นจึงอ่านค่าจากรีจิสเตอร์ที่ใช้ในการประมวลผลในสเตจถัดไป

4.3 สเตจ EX: Execute (ประมวลผล) ➕➖

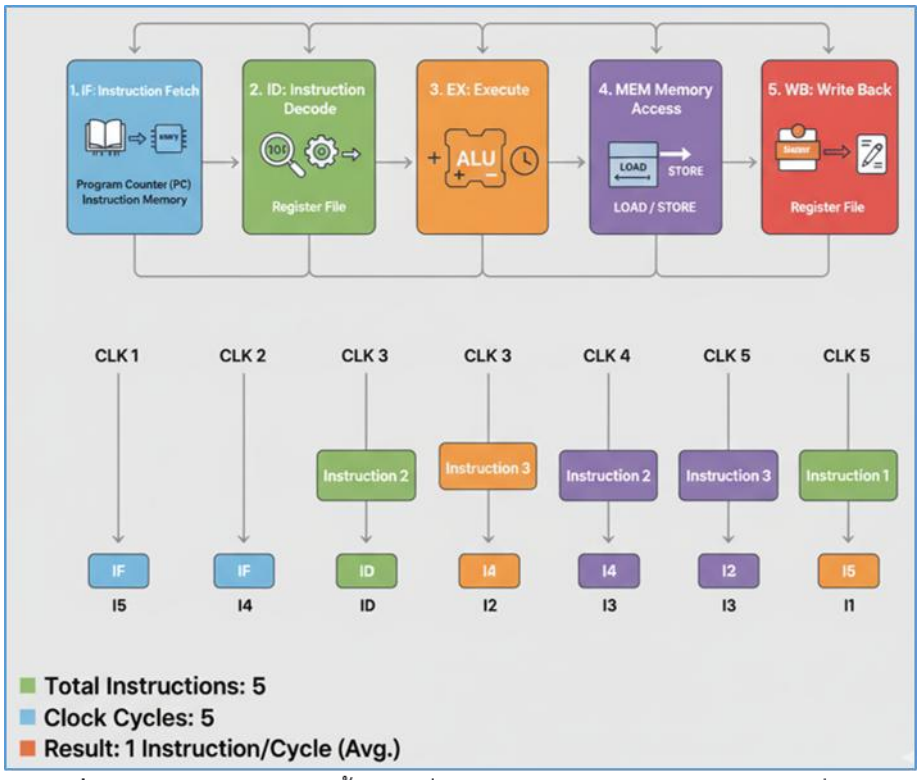
- **หน้าที่:** ดำเนินการคำนวณตามคำสั่งที่ถอดรหัสมา โดยใช้หน่วยประมวลผลเลขคณิตและตรรกะ (ALU - Arithmetic and Logic Unit)
- **การทำงาน:** หากเป็นคำสั่งทางคณิตศาสตร์ (เช่น ADD, SUB), ALU จะทำการบวกหรือลบตามที่คำสั่งระบุ หากเป็นคำสั่งที่ต้องเข้าถึงหน่วยความจำ (เช่น LOAD, STORE), สเตจนี้จะคำนวณที่อยู่ของหน่วยความจำที่จะต้องเข้าถึงในสเตจต่อไป

4.4 สเตจ MEM: Memory Access (เข้าถึงหน่วยความจำ) 💾

- **หน้าที่:** เข้าถึงหน่วยความจำข้อมูล (Data Memory) เพื่ออ่านหรือเขียนข้อมูล
- **การทำงาน:**
    - ถ้าเป็นคำสั่ **LOAD** (โหลดข้อมูล) จะอ่านข้อมูลจากที่อยู่ที่คำนวณไว้ในสเตจ EX
    - ถ้าเป็นคำสั่ **STORE** (เก็บข้อมูล) จะเขียนข้อมูลลงในที่อยู่ที่คำนวณไว้ในสเตจ EX

4.5 สเตจ WB: Write Back (เขียนผลลัพธ์กลับ)

- **หน้าที่:** เขียนผลลัพธ์สุดท้ายของการประมวลผลกลับไปยังรีจิสเตอร์ (Register File)



รูปที่ 8 – 7 แผนภาพแสดงขั้นตอนย่อย 5 สถานะในการประมวลผลคำสั่งของสถาปัตยกรรมไปป์ไลน์แบบ RISC

- **การทำงาน:** ค่าที่ได้จากการคำนวณในสเตจ EX หรือค่าที่อ่านได้จากหน่วยความจำในสเตจ MEM จะถูกเขียนกลับไปยังรีจิสเตอร์ปลายทางที่ระบุไว้ในคำสั่ง เพื่อให้คำสั่งถัดไปสามารถนำไปใช้ได้

รูปแบบการทำงานจริงแบบไปป์ไลน์ การประมวลผลคำสั่ง 3 คำสั่งในไปป์ไลน์ 5 สเตจ เพื่อให้เห็นภาพการทำงานพร้อมกัน:

รอบสัญญาณนาฬิกา	**สเตจ IF** (ดึง)	**สเตจ ID** (ถอด)	**สเตจ EX** (ประมวล)	**สเตจ MEM** (หน่วยความจำ)	**สเตจ WB** (เขียนกลับ)
รอบที่ 1	คำสั่ง 1 (IF)				
รอบที่ 2	คำสั่ง 2 (IF)	คำสั่ง 1 (ID)			
รอบที่ 3	คำสั่ง 3 (IF)	คำสั่ง 2 (ID)	คำสั่ง 1 (EX)		
รอบที่ 4		คำสั่ง 3 (ID)	คำสั่ง 2 (EX)	คำสั่ง 1 (MEM)	
รอบที่ 5			คำสั่ง 3 (EX)	คำสั่ง 2 (MEM)	คำสั่ง 1 (WB)
รอบที่ 6				คำสั่ง 3 (MEM)	คำสั่ง 2 (WB)
รอบที่ 7					คำสั่ง 3 (WB)

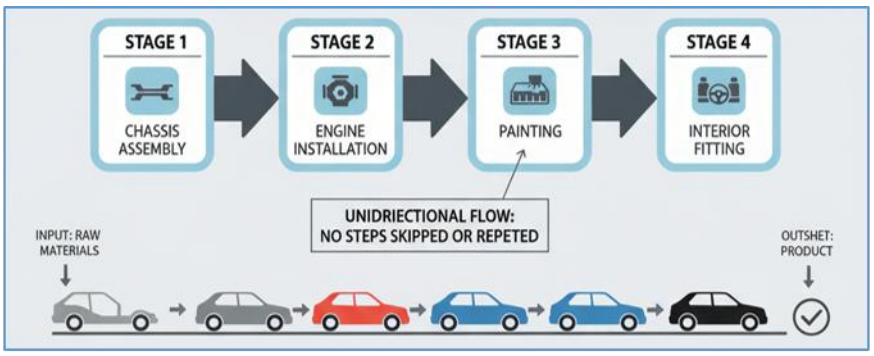
ตารางที่ 8-4 ขั้นตอนไทม์ไลน์การทำงานของคำสั่งในแต่ละรอบสัญญาณนาฬิกาบนระบบ Pipeline

จากตารางนี้จะเห็นว่าใน **รอบที่ 3** CPU สามารถทำงานได้ถึง 3 คำสั่งพร้อมกันในแต่ละสเตจ ซึ่งเป็นการเพิ่มประสิทธิภาพได้อย่างชัดเจน แม้ว่าคำสั่งที่ 1 จะใช้เวลา 5 รอบสัญญาณนาฬิกาในการประมวลผลทั้งหมด แต่คำสั่งถัดไปจะเสร็จสิ้นในแต่ละรอบสัญญาณนาฬิกาหลังจากนั้น ทำให้ได้ผลลัพธ์ที่รวดเร็วขึ้นมาก

5.การทำงานไปป์ไลน์แบบเป็นลำดับ (Linear Pipeline) และแบบไม่เป็นลำดับ (Non-linear Pipeline)

5.1 ไปป์ไลน์แบบเป็นลำดับ (Linear Pipeline)

แนวคิด: เป็นไปป์ไลน์ที่ **ทำงานต่อเนื่องกันไปในทิศทางเดียว** โดยที่ผลลัพธ์จากขั้นตอนหนึ่งจะถูกส่งต่อไปยังขั้นตอนถัดไปเท่านั้น และไม่มีการทำงานย้อนกลับหรือข้ามขั้นตอน เป็นรูปแบบที่นิยมใช้มากที่สุดใน CPU ทั่วไปเพราะมีโครงสร้างที่เรียบง่ายและจัดการได้ง่าย



รูปที่ 8 – 8 ภาพจำลองลักษณะการไหลของข้อมูลในระบบไปป์ไลน์ทิศทางเดียว (Unidirectional Flow)

การทำงาน:

- แต่ละขั้นตอน (Stage) มีหน้าที่เฉพาะเจาะจง
- คำสั่งจะถูกป้อนเข้าสู่ขั้นตอนแรก (Input Stage)
- ข้อมูลจะไหลจากขั้นตอนหนึ่งไปยังอีกขั้นตอนหนึ่งอย่างเป็นลำดับ
- เมื่อขั้นตอนสุดท้าย (Output Stage) เสร็จสิ้น ข้อมูลก็จะออกจากไปป์ไลน์

ตัวอย่าง: ลองนึกถึงโรงงานผลิตรถยนต์ที่แบ่งงานออกเป็นสถานีต่าง ๆ:

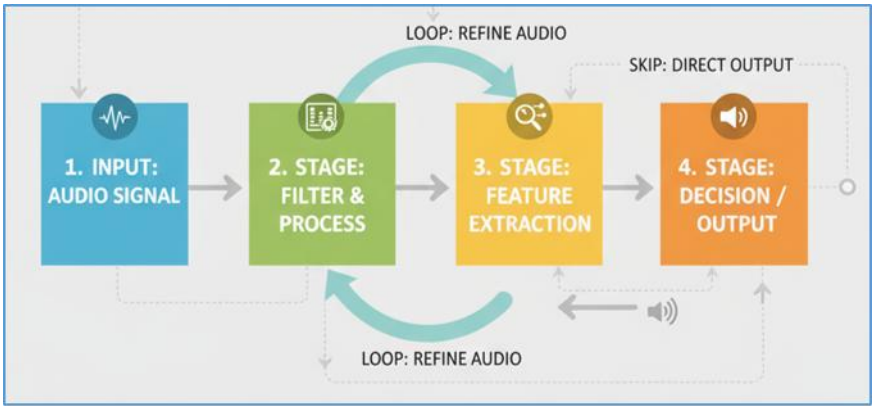
1. **สถานีที่ 1 (ประกอบโครงสร้าง):** รับชิ้นส่วนโครงสร้างรถ
2. **สถานีที่ 2 (ติดตั้งเครื่องยนต์):** รับโครงสร้างที่ประกอบเสร็จแล้วจากสถานี 1
3. **สถานีที่ 3 (ทำสี):** รับตัวถังที่มีเครื่องยนต์แล้วจากสถานี 2
4. **สถานีที่ 4 (ติดตั้งภายใน):** รับรถที่ทำสีแล้วจากสถานี 3 รถแต่ละคันจะไหลไปตามสถานีต่าง ๆ อย่างต่อเนื่อง ไม่มีการกลับไปทำสถานีเดิมอีก

5.2 ไปป์ไลน์แบบไม่เป็นลำดับ (Non-linear Pipeline)

แนวคิด: เป็นไปป์ไลน์ที่มีความซับซ้อนกว่า โดย **อนุญาตให้มีการทำงานแบบข้ามไปข้ามมา** หรือมีวงจรย้อนกลับ (feedback loop) เพื่อนำผลลัพธ์บางส่วนกลับมาประมวลผลซ้ำในขั้นตอนก่อนหน้า หรือข้ามบางขั้นตอนที่ไม่จำเป็น เพื่อให้การประมวลผลมีความยืดหยุ่นมากขึ้น

การทำงาน:

- มีการเชื่อมต่อแบบซับซ้อนระหว่างขั้นตอน และ อาจมีหลายเส้นทาง (multiple paths) ในการประมวลผลคำสั่งหนึ่ง ๆ สำหรับใช้ในการประมวลผลสัญญาณดิจิทัล (Digital Signal Processing - DSP) หรือสถาปัตยกรรมแบบพิเศษ

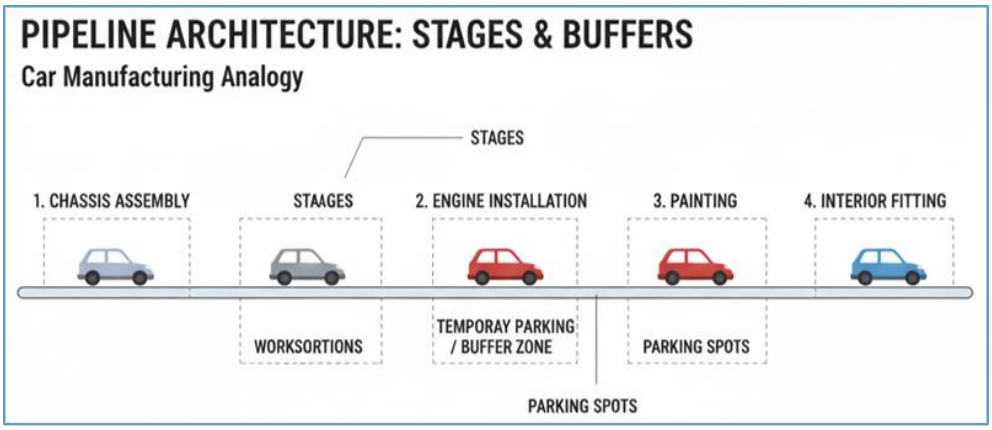


รูปที่ 8 – 9 ภาพจำลองลักษณะการไหลของข้อมูลในระบบไปป์ไลน์แบบมีเงื่อนไขป้อนกลับและข้ามขั้นตอน

ตัวอย่าง:

- **การคำนวณซ้ำ:** ในการประมวลผลสัญญาณเสียง อาจต้องมีการวนลูปกลับไปประมวลผลในขั้นตอนก่อนหน้า เพื่อปรับปรุงคุณภาพเสียงให้ดีขึ้น
- **การประมวลผลภาพแบบปรับตัว (Adaptive Image Processing):** ข้อมูลภาพอาจถูกส่งไปประมวลผลในขั้นตอนหนึ่งเพื่อหาขอบภาพ จากนั้นผลลัพธ์จะถูกส่งกลับไปให้ขั้นตอนแรกเพื่อประมวลผลส่วนอื่น ๆ ของภาพอีกครั้ง โดยอาศัยข้อมูลจากขั้นตอนที่แล้ว

5.3 การทำงานโครงสร้างของไปป์ไลน์ (Pipeline Architecture)



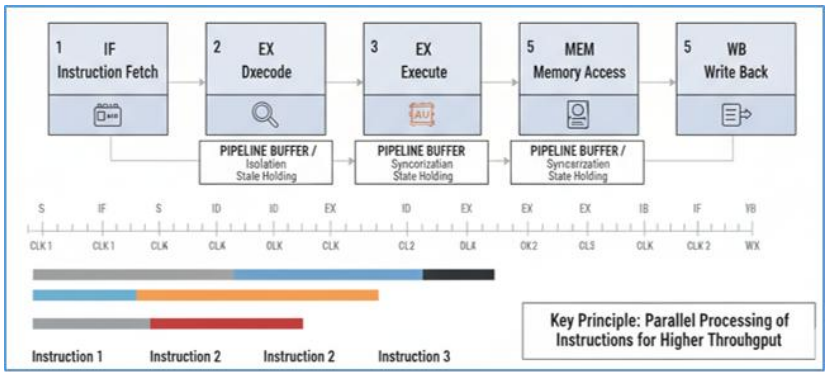
รูปที่ 8 – 10 ภาพจำลองโครงสร้างสถาปัตยกรรมไปป์ไลน์ที่ประกอบด้วยสถานีประมวลผล (Stages) และหน่วยพักข้อมูล (Buffers)

ไปป์ไลน์ (Pipeline) เป็นเทคนิคที่ใช้ในหน่วยประมวลผลเพื่อเพิ่มความเร็วในการประมวลผลโดยรวม โครงสร้างหลักของไปป์ไลน์ประกอบด้วย **สเตจ (Stage)** ซึ่งเป็นขั้นตอนการทำงานย่อย ๆ และ **บัฟเฟอร์ (Buffer)** หรือ **แลตช์ (Latch)** ซึ่งทำหน้าที่เป็นหน่วยเก็บข้อมูลชั่วคราวระหว่างสเตจ

โครงสร้างของไปป์ไลน์ (Pipeline Architecture) เหมือนกับ **สายพานการผลิตรถยนต์**

- **สเตจ (Stage):** คือ **สถานีงาน** ต่าง ๆ บนสายพานการผลิต
- **บัฟเฟอร์ (Buffer)/แลตช์ (Latch):** คือ **พื้นที่พักรถชั่วคราว** ระหว่างสถานีงานแต่ละแห่ง

ในทางสถาปัตยกรรมคอมพิวเตอร์ โครงสร้างไปป์ไลน์จะประกอบไปด้วยหลายสเตจที่เรียงต่อกัน แต่ละสเตจจะทำหน้าที่ประมวลผลคำสั่งในส่วนของตัวเอง



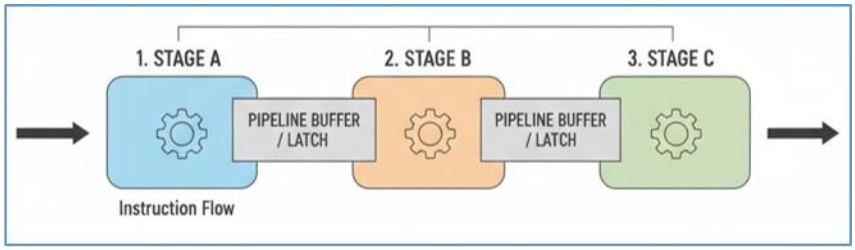
รูปที่ 8 – 11 แผนภาพหลักการประมวลผลคำสั่งแบบคู่ขนานเพื่อเพิ่มอัตราการส่งผ่านข้อมูล (Throughput)

5.3.1 สเตจ (Stage) : คือ **ขั้นตอนย่อยของการประมวลผลคำสั่ง** โดยในแต่ละรอบสัญญาณนาฬิกา (Clock Cycle) แต่ละสเตจจะประมวลผลคำสั่งที่แตกต่างกันไป ตัวอย่างที่พบบ่อยใน CPU แบบไปป์ไลน์ 5 สเตจ ได้แก่:

1. **สเตจ Fetch (IF):** ดึงคำสั่งจากหน่วยความจำ
2. **สเตจ Decode (ID):** ถอดรหัสคำสั่งและอ่านข้อมูลจากรีจิสเตอร์ที่เกี่ยวข้อง
3. **สเตจ Execute (EX):** ประมวลผลคำสั่งทางคณิตศาสตร์หรือตรรกะ
- **4. สเตจ Memory Access (MEM):** เข้าถึงหน่วยความจำเพื่ออ่านหรือเขียนข้อมูล
- **5. สเตจ Write Back (WB):** เขียนผลลัพธ์สุดท้ายกลับไปยังรีจิสเตอร์

การมีหลายสเตจช่วยให้ CPU สามารถประมวลผลหลายคำสั่งพร้อมกันได้ เช่น ขณะที่คำสั่งที่ 1 กำลังอยู่ในสเตจ Write Back, คำสั่งที่ 2 ก็อาจจะอยู่ในสเตจ Execute และคำสั่งที่ 3 กำลังอยู่ในสเตจ Fetch ทำให้โดยรวมแล้ว CPU ทำงานได้เร็วขึ้นมาก

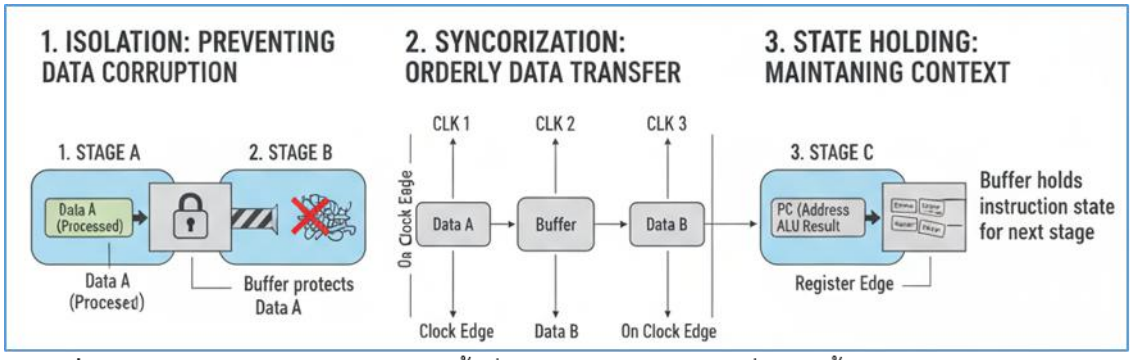
5.3.2 บัฟเฟอร์ (Buffer) หรือ แลตช์ (Latch) คือ **หน่วยเก็บข้อมูลชั่วคราว** ที่อยู่**ระหว่างสเตจ** แต่ละสเตจใน



รูปที่ 8 – 12 แผนภาพจำลองการเชื่อมต่อระหว่างสถานีประมวลผลด้วยหน่วยพักข้อมูล (Pipeline Buffer / Latch)

ไปป์ไลน์ มีบทบาทสำคัญอย่างมากในการทำงานดังนี้:

1. **ป้องกันข้อมูลปนกัน (Isolation):** เพื่อให้แน่ใจว่าผลลัพธ์ที่ประมวลผลเสร็จแล้วจากสเตจก่อนหน้าจะไม่ถูกเปลี่ยนแปลงโดยสเตจถัดไป
2. **ส่งข้อมูลอย่างต่อเนื่อง (Synchronization):** เพื่อส่งข้อมูลจากสเตจหนึ่งไปยังอีกสเตจหนึ่งอย่างเป็นระเบียบและพร้อมเพรียงกันในแต่ละรอบสัญญาณนาฬิกา
3. **เก็บสถานะ (Holding State):** เพื่อเก็บข้อมูลและสถานะของคำสั่งที่กำลังไหลผ่านไปป์ไลน์

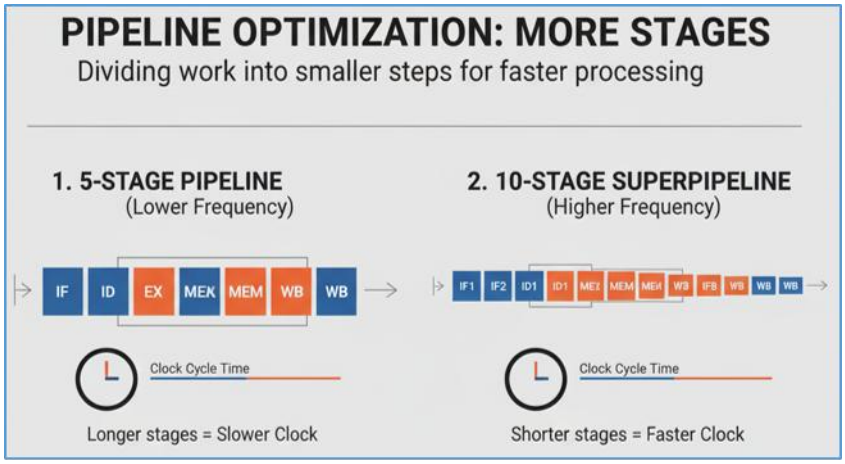


รูปที่ 8 – 13 แผนภาพแสดงบทบาทหน้าที่ 3 ประการหลักของหน่วยพักข้อมูล (Pipeline Buffer) ในระบบไปป์ไลน์

สรุปการเปรียบเทียบกับโรงงานผลิตรถยนต์:

- **สเตจ** คือ สถานีงานแต่ละแห่ง (เช่น สถานีทำสี) ที่ใช้เวลาทำงานตามรอบที่กำหนด
- **บัฟเฟอร์** คือ พื้นที่จอดรถชั่วคราวระหว่างสถานีทำสีและสถานีติดตั้งภายใน รถยนต์จะถูกจอดพักไว้ที่นี่หลังจากทำสีเสร็จแล้ว และจะถูกส่งต่อไปยังสถานีถัดไปเมื่อสถานีนั้นว่าง

การมีบัฟเฟอร์ทำให้แต่ละสเตจสามารถทำงานได้อย่างเป็นอิสระจากสเตจอื่น ๆ เมื่อสเตจหนึ่งทำงานเสร็จ ผลลัพธ์จะถูกเก็บไว้ในบัฟเฟอร์ และสเตจนั้นก็สามารถรับคำสั่งใหม่มาประมวลผลได้ทันทีโดยไม่ต้องรอให้คำสั่งก่อนหน้าประมวลผลเสร็จสิ้นทุกขั้นตอน



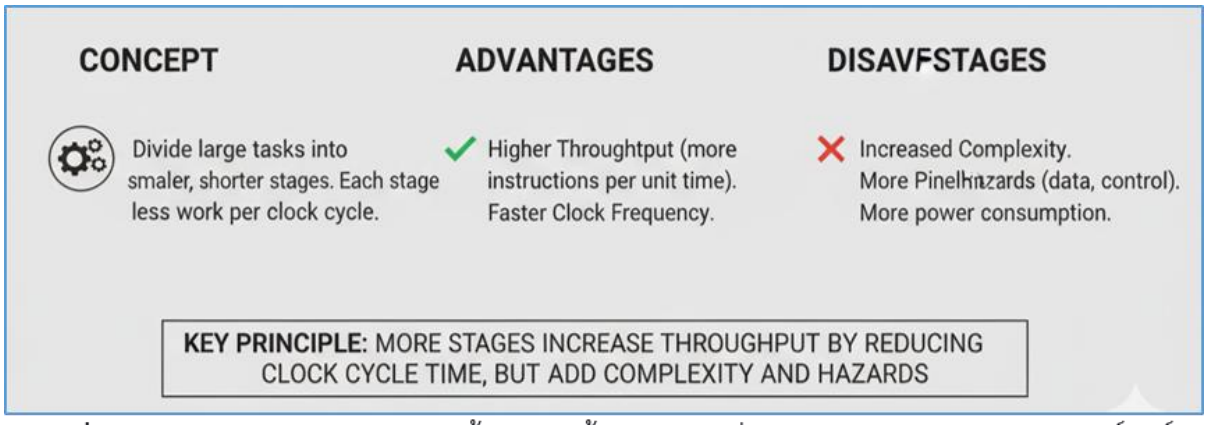
รูปที่ 8 – 14 แผนภาพเปรียบเทียบประสิทธิภาพระหว่างสถาปัตยกรรมไปป์ไลน์แบบ 5 ขั้นตอน และ 10 ขั้นตอน (Super pipeline)

5.การการเพิ่มประสิทธิภาพของไปป์ไลน์ (Pipeline Optimization)

เพื่อเพิ่มประสิทธิภาพของไปป์ไลน์ (Pipeline) หรือให้ประมวลผลได้เร็วขึ้น สามารถทำได้โดยการปรับปรุงปัจจัยหลัก 2 อย่าง คือ **การเพิ่มจำนวนสเตจ** และ **การลดระยะเวลาของแต่ละสเตจ** ซึ่งทั้งสองวิธีนี้จะช่วยเพิ่มอัตราการประมวลผลโดยรวมของระบบ

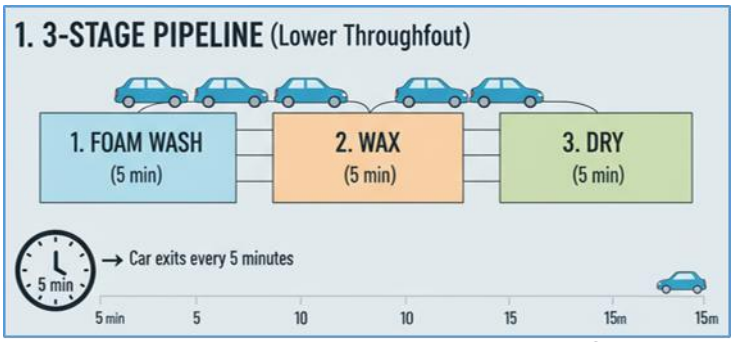
1. การเพิ่มจำนวนสเตจ (More Stages) : แนวคิดคือการแบ่งงานใหญ่ให้เป็นงานย่อย ๆ มากขึ้น เพื่อให้แต่ละส่วนงานมีขนาดเล็กลงและใช้เวลาน้อยลงในการทำงาน ทำให้สามารถส่งงานไปยังสเตจถัดไปได้เร็วขึ้น

- **คำอธิบาย:** การเพิ่มสเตจไปป์ไลน์จาก 5 สเตจ เป็น 10 หรือ 20 สเตจ จะช่วยลดเวลาที่ใช้ในแต่ละรอบสัญญาณนาฬิกา (Clock Cycle Time) ของไปป์ไลน์ลงได้จากงานที่ต้องทำในแต่ละสเตจสั้นลง ส่งผลให้ความถี่ (Frequency) ของ CPU เพิ่มขึ้น และทำให้สามารถประมวลผลคำสั่งได้มากขึ้นต่อหน่วยเวลา
- **ข้อดี:** เพิ่ม **Throughput** (จำนวนงานที่ทำเสร็จในหนึ่งหน่วยเวลา) ของระบบอย่างมีนัยสำคัญ
- **ข้อเสีย:** การเพิ่มสเตจมากเกินไปอาจทำให้เกิดปัญหา **Pipeline Hazards** (ความเสี่ยงในไปป์ไลน์) ได้มากขึ้น รวมถึงความซับซ้อนในการควบคุมและจัดการไปป์ไลน์ก็เพิ่มขึ้นด้วย



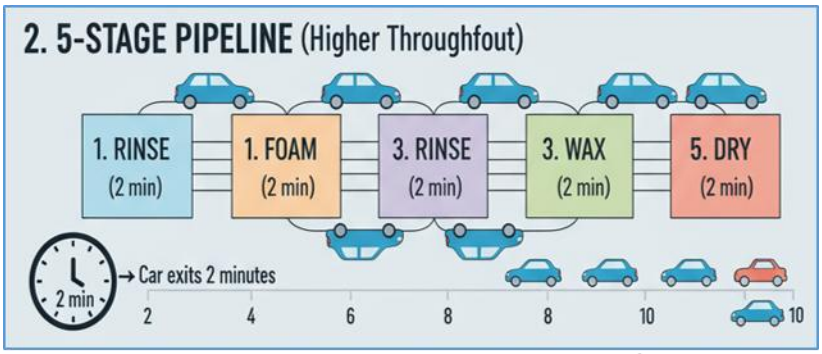
รูปที่ 8 – 15 แผนภาพสรุปแนวคิด ข้อดี และข้อเสีย การเพิ่มสถานีประมวลผลในระบบไปป์ไลน์

ตัวอย่างเปรียบเทียบ ลองนึกถึงการล้างรถ



รูปที่ 8 – 16 ภาพจำลองระบบไปป์ไลน์แบบ 3 ขั้นตอน (3-Stage Pipeline) ด้วยกิจกรรมคาร์แคร์

- **ไปป์ไลน์แบบ 3 สเตจ:**
  - **สเตจ 1:** ล้างโฟม (5 นาที)
  - **สเตจ 2:** ขัดเงา (5 นาที)
  - **สเตจ 3:** เช็ดแห้ง (5 นาที)



รูปที่ 8 – 17 ภาพจำลองระบบไปป์ไลน์แบบ 5 ขั้นตอน (5-Stage Pipeline) เพื่อเพิ่มอัตราการส่งผ่านข้อมูล

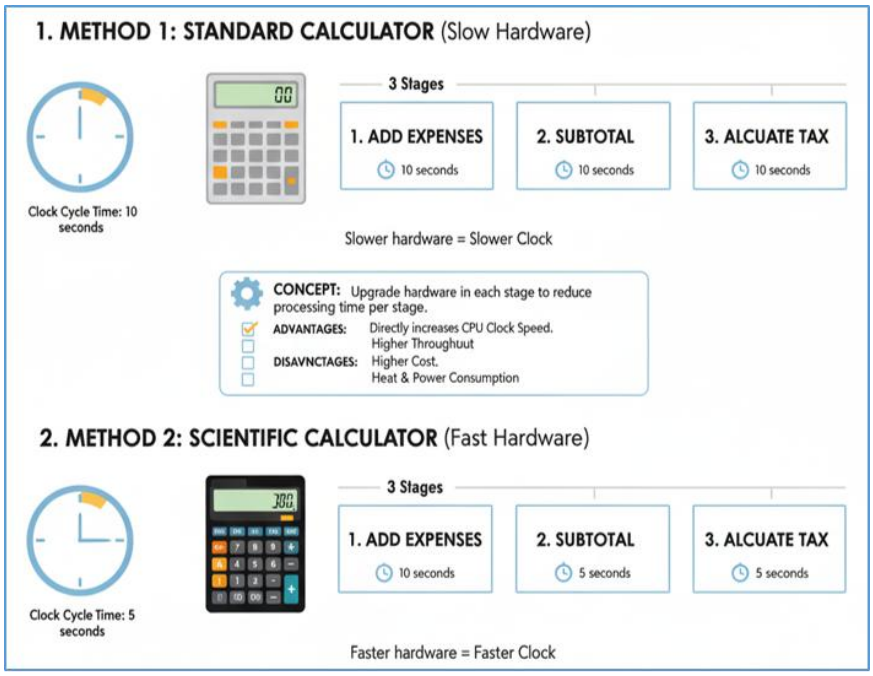
- **ไปป์ไลน์แบบ 5 สเตจ:**
  - **สเตจ 1:** ฉีดน้ำ (2 นาที)
  - **สเตจ 2:** ล้างโฟม (2 นาที)
  - **สเตจ 3:** ฉีดน้ำล้างโฟม (2 นาที)
  - **สเตจ 4:** ขัดเงา (2 นาที)
  - **สเตจ 5:** เช็ดแห้ง (2 นาที)

แม้ว่าเวลาที่ใช้ในการล้างรถแต่ละคัน (Latency) จะเท่ากัน แต่แบบ 5 สเตจจะสามารถส่งรถออกจากไปป์ไลน์ได้ทุก ๆ 2 นาที ขณะที่แบบ 3 สเตจต้องรอ 5 นาที ซึ่งทำให้แบบ 5 สเตจล้างรถได้จำนวนมากกว่าในระยะเวลาเท่ากัน

2. การลดระยะเวลาของแต่ละสเตจ (Reducing Stage Time)

แนวคิดคือการทำให้แต่ละขั้นตอนย่อยเสร็จเร็วขึ้น ซึ่งทำได้โดยการปรับปรุงฮาร์ดแวร์ที่ใช้ในแต่ละสเตจ

- **คำอธิบาย:** ในแต่ละสเตจของไปป์ไลน์จะมีฮาร์ดแวร์เฉพาะสำหรับงานนั้น ๆ เช่น สเตจ Execute มีหน่วยประมวลผลเลขคณิตและตรรกะ (ALU) การใช้ ALU ที่มีความเร็วสูงกว่าหรือมีประสิทธิภาพมากกว่าจะช่วยลดเวลาที่ใช้ในสเตจนั้น ๆ ได้
- **ข้อดี:** ส่งผลให้ความเร็วของรอบสัญญาณนาฬิกา (Clock Speed) ของ CPU เพิ่มขึ้นโดยตรง
- **ข้อเสีย:** การปรับปรุงฮาร์ดแวร์ให้เร็วขึ้นมักจะมีต้นทุนสูงกว่า และอาจทำให้เกิดปัญหาเรื่องความร้อนและการใช้พลังงานมากขึ้น



รูปที่ 8 – 18 แผนภาพเปรียบเทียบประสิทธิภาพไปป์ไลน์ระหว่างฮาร์ดแวร์ความเร็วต่ำและฮาร์ดแวร์ความเร็วสูง (Hardware Upgrade)

ตัวอย่างเปรียบเทียบ การทำงานคำนวนบัญชีค่าใช้จ่าย

- **วิธีที่ 1:** ใช้เครื่องคิดเลขแบบธรรมดา (แต่ละขั้นตอนใช้เวลา 10 วินาที)
- **วิธีที่ 2:** ใช้เครื่องคิดเลขแบบวิทยาศาสตร์ที่คำนวณเร็วขึ้น (แต่ละขั้นตอนใช้เวลา 5 วินาที)

แม้ว่าจำนวนขั้นตอนจะเท่ากัน แต่การใช้เครื่องมือที่เร็วขึ้นจะช่วยให้งานทั้งหมดเสร็จเร็วขึ้นได้

สรุปในทางปฏิบัติ การเพิ่มประสิทธิภาพของไปป์ไลน์มักจะใช้ทั้งสองวิธีร่วมกัน เพื่อให้ได้ความเร็วในการประมวลผลสูงสุดเท่าที่จะเป็นไปได้

7.ปัญหาในการทำงานแบบไปป์ไลน์ (Pipeline Hazards):

การทำงานแบบไปป์ไลน์ แม้จะช่วยเพิ่มประสิทธิภาพ แต่ก็มีปัญหาที่ทำให้การทำงานไม่ราบรื่น ซึ่งเราเรียกว่า **Pipeline Hazards** หรือ **ความเสี่ยงของไปป์ไลน์** ปัญหาเหล่านี้ทำให้ไปป์ไลน์ต้อง "หยุดชะงัก" หรือ "ล้างคำสั่งที่ดึงมาล่วงหน้า" ซึ่งส่งผลกระทบต่อความเร็วในการประมวลผลโดยรวมแบ่งได้ 3 แบบ

7.1 Hazard แบบข้อมูล (Data Hazard) เกิดขึ้นเมื่อคำสั่งหนึ่งต้องการ **ข้อมูล (Data)** ที่ได้จากคำสั่งก่อนหน้าที่ยังประมวลผลไม่เสร็จ ทำให้คำสั่งถัดไปต้อง **รอ (Stall)** จนกว่าข้อมูลจะพร้อมใช้งาน

ลองนึกถึงสถานการณ์ง่ายๆ: คุณกำลังทำอาหาร

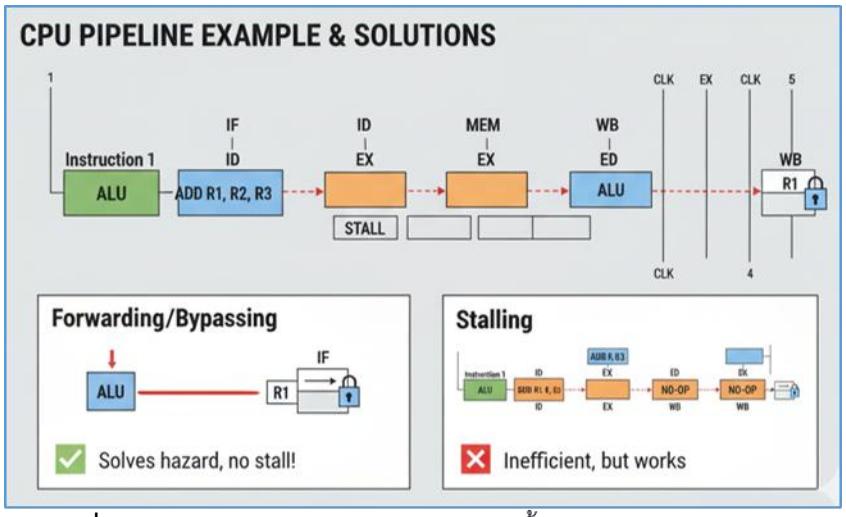
- **คำสั่งที่ 1:** ทอดไข่เจียว
- **คำสั่งที่ 2:** ทำข้าวผัดไข่



รูปที่ 8 – 19 ภาพจำลองสภาวะติดขัดเชิงข้อมูล (Data Hazard) ในระบบไปป์ไลน์ด้วยกิจกรรมการทำอาหาร

คุณจะเริ่มทำข้าวผัดไข่ไม่ได้ ถ้าไข่เจียว (ข้อมูลที่ต้องการ) ยังทอดไม่เสร็จและนำมาใช้ไม่ได้ นั่นคือ **Data Hazard** ในทางคอมพิวเตอร์ เกิดขึ้นเมื่อคำสั่งที่สองพยายามอ่านค่าจากรีจิสเตอร์ (Register) ที่คำสั่งแรกกำลังจะเขียนผลลัพธ์กลับเข้าไป แต่คำสั่งแรกยังเขียนไม่เสร็จ

ในภาพนี้ คำสั่ง ADD คำนวณเสร็จในสเตจ EX แต่จะเขียนผลลัพธ์กลับไปยังรีจิสเตอร์ 1 ในสเตจ WB คำสั่ง SUB ต้องการค่าจากรีจิสเตอร์ 1 ในสเตจ ID แต่ค่านั้นยังไม่ถูกเขียนกลับเข้ามา ทำให้ SUB ต้องหยุดรอ



รูปที่ 8 – 20 แผนภาพแสดงกระบวนการแก้ไขปัญหา Data Hazard ด้วยวิธี Forwarding/Bypassing และวิธี Stalling

วิธีแก้ปัญหา:

- **Forwarding/Bypassing:** ส่งผลลัพธ์จากสเตจ EX ของคำสั่งแรกไปยังสเตจ ID ของคำสั่งที่สองโดยตรง โดยไม่ต้องรอให้เขียนกลับเข้าไปในรีจิสเตอร์ก่อน
- **Stalling:** หยุดการทำงานของไปป์ไลน์ชั่วคราว จนกว่าข้อมูลจะพร้อม

<u>7.2. Hazard แบบโครงสร้าง (Structural Hazard)</u> เกิดขึ้นเมื่อ **ทรัพยากร (Resource)** ในระบบมีจำกัด ทำให้คำสั่งหลายคำสั่งต้องการใช้ทรัพยากรเดียวกันพร้อมกันในเวลาเดียวกัน

ตัวอย่าง: ในโรงงานผลิตรถยนต์ มีเครื่องทำสีแค่เครื่องเดียว

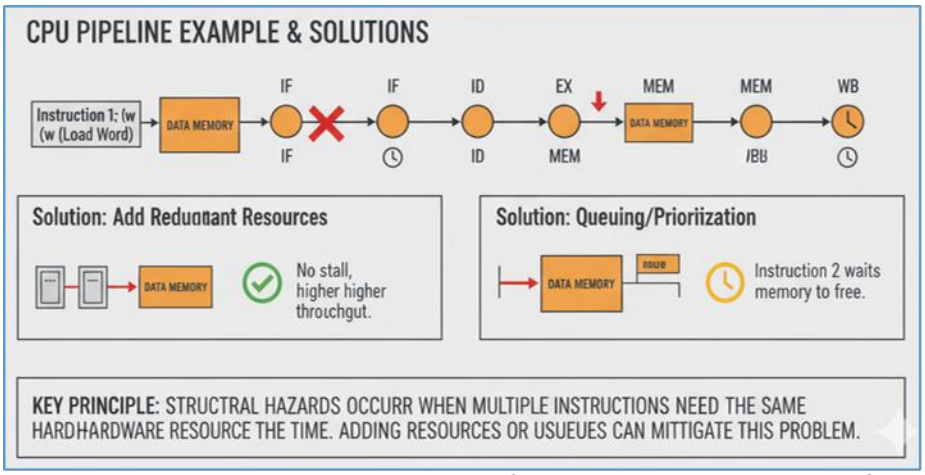
รถคันที่ 1 เข้ามาใช้เครื่องทำสี รถคันที่ 2 ก็ต้องรอ เพราะเครื่องทำสีไม่ว่าง



รูปที่ 8 – 21 ภาพจำลองสภาวะติดขัดเชิงโครงสร้าง (Structural Hazard) ในระบบไปป์ไลน์ด้วยแบบจำลองโรงงานผลิตรถยนต์

ใน CPU ที่ใช้ไปป์ไลน์ อาจเกิดปัญหาเมื่อมีคำสั่งสองคำสั่งที่ต้องการใช้ **หน่วยคำนวณและตรรกะ (ALU)** พร้อมกัน หรือต้องการใช้พอร์ตหน่วยความจำพร้อมกัน ซึ่งทำให้เกิดการชนกันของทรัพยากร

รูปที่ 8–22 ในภาพนี้มีทั้งคำสั่ง lw (Load Word) และคำสั่ง sw (Store Word) ต่างก็ต้องใช้หน่วยความจำ



รูปที่ 8 – 22 แผนภาพแสดงกระบวนการแก้ไขปัญหา Structural Hazard ด้วยวิธีเพิ่มทรัพยากร (Add Redundant Resources) และวิธีจัดคิว (Queuing/Prioritization)

ในสเตจ MEM และ ถ้าทั้งสองคำสั่งมาถึงพร้อมกันในสเตจ MEM จะเกิด **Structural Hazard** เพราะมีพอร์ตหน่วยความจำเพียงพอร์ตเดียว

วิธีแก้ปัญหา:

- **เพิ่มทรัพยากร:** เพิ่มจำนวนทรัพยากรที่จำกัด เช่น มี ALU สองชุด หรือพอร์ตหน่วยความจำสองพอร์ต
- **จัดการคิว:** ใช้การจัดคิวเพื่อให้คำสั่งหนึ่งทำงานก่อนและอีกคำสั่งหนึ่งรอ

ปัญหาในการทำงานแบบไปป์ไลน์ (Pipeline Hazards) แบบควบคุม (Control Hazard)

<u>7.3.Hazard แบบควบคุม (Control Hazard)</u> เกิดขึ้นเมื่อมีการใช้คำสั่งที่ต้อง **เปลี่ยนทิศทางการทำงาน** ของโปรแกรม เช่น คำสั่ง **if**, **jump**, หรือ **goto** เนื่องจากไปป์ไลน์มีการดึงคำสั่งล่วงหน้า (Pre-fetching) แต่ไม่รู้ว่าเงื่อนไขของคำสั่ง jump จะเป็นจริงหรือไม่ ทำให้ไปป์ไลน์อาจดึงคำสั่งผิดมาและต้องล้างทิ้งทั้งหมด

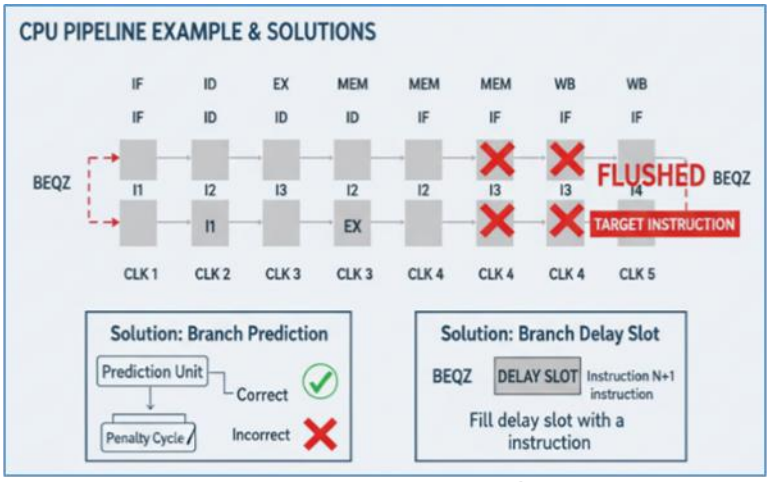
ตัวอย่าง: คุณกำลังอ่านแผนที่และเตรียมเดินทาง

- **แผนที่:** "ถ้าเจอแยกไฟแดงเลี้ยวซ้าย (ถ้าเป็นจริง) หรือตรงไป (ถ้าเป็นเท็จ)"



รูปที่ 8 – 23 ภาพจำลองสภาวะติดขัดเชิงการควบคุม (Control Hazard) จากการทำนายเงื่อนไขผิดพลาด

- คุณไม่รู้ว่าแยกไฟแดงจะอยู่ข้างหน้าหรือไม่ คุณจึงเดาและเตรียมตัวเลี้ยวซ้ายไว้ก่อน แต่พอไปถึงจริงๆ คุณต้องตรงไป ทำให้การเตรียมตัวก่อนหน้าสูญเปล่า



รูปที่ 8 – 24 แผนภาพแสดงกระบวนการแก้ไขปัญหา Control Hazard ด้วยวิธีทำนายผลสาขา (Branch Prediction) และวิธีลดความล่าช้า (Delayed Branch)

ใน CPU ก็เช่นกัน เมื่อเจอคำสั่ง jump ไปป์ไลน์จะเดาและดึงคำสั่งถัดไปมาประมวลผลล่วงหน้า แต่ถ้าการเดาผิด คำสั่งที่ดึงมาล่วงหน้าจะต้องถูก **ล้างทิ้ง (Flush)** ทั้งหมด ทำให้เสียเวลาไปโดยเปล่าประโยชน์

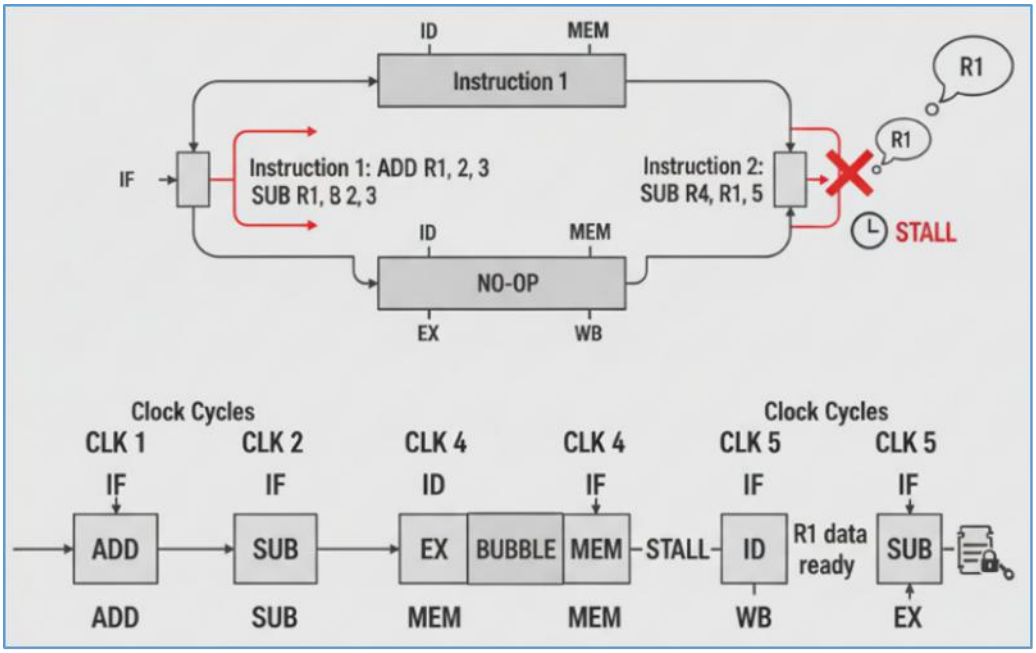
ภาพประกอบ: ในภาพนี้: คำสั่ง BEQZ (Branch if Equal to Zero) จะตัดสินใจในสเตจ ID ว่าจะกระโดดไปที่อยู่ใหม่หรือไม่ และ ในขณะเดียวกัน ไปป์ไลน์ก็ดึงคำสั่ง I3 และ I4 มาล่วงหน้าแล้ว ถ้าเงื่อนไข BEQZ เป็นจริง ไปป์ไลน์จะต้องยกเลิกคำสั่ง I3 และ I4 ทั้งหมด และไปดึงคำสั่งที่อยู่ใหม่มาแทน

วิธีแก้ปัญหา:

- **Branch Prediction:** ใช้เทคนิคการคาดเดาว่าคำสั่ง jump จะไปในทิศทางไหน
- **Branch Delay Slot:** วางคำสั่งที่สามารถประมวลผลได้เสมอไว้หลังคำสั่ง jump เพื่อให้ไปป์ไลน์ทำงานได้ในขณะที่รอผลลัพธ์
- **Stalling:** หยุดไปป์ไลน์ชั่วคราวจนกว่าผลลัพธ์ของคำสั่ง jump จะพร้อม

7.7 เทคนิคการแก้ปัญหา Hazard แบบการหยุดรอ (Stalling) หรือ Bubble

Hazard หรือปัญหาในไปป์ไลน์เป็นสิ่งที่ทำให้การทำงานสะดุดลงได้ แต่ก็มีหลายเทคนิคที่ถูกพัฒนาขึ้นมาเพื่อแก้ไขปัญหาเหล่านี้ ทำให้ไปป์ไลน์สามารถทำงานได้อย่างมีประสิทธิภาพสูงสุดเท่าที่จะเป็นไปได้



รูปที่ 8 – 25 แผนภาพจำลองการทำงานของคำสั่งว่าง (NO-OP) และสภาวะฟองอากาศ (Bubble) เพื่อแก้ไขปัญหา Data Hazard

Stalling คือการ **แทรกขั้นตอนว่างๆ** ที่เรียกว่า **"Bubble" (ฟองอากาศ)** หรือ **No-Op (No Operation)** เข้าไปในไปป์ไลน์ เพื่อให้คำสั่งที่ต้องการข้อมูลต้องหยุดรอชั่วคราว จนกว่าข้อมูลจากคำสั่งก่อนหน้าจะพร้อมใช้งาน เทคนิคนี้เป็นการแก้ปัญหาที่เรียบง่ายแต่ทำให้ประสิทธิภาพของไปป์ไลน์ลดลง เนื่องจากมีการเสียรอบสัญญาณนาฬิกาไปโดยเปล่าประโยชน์

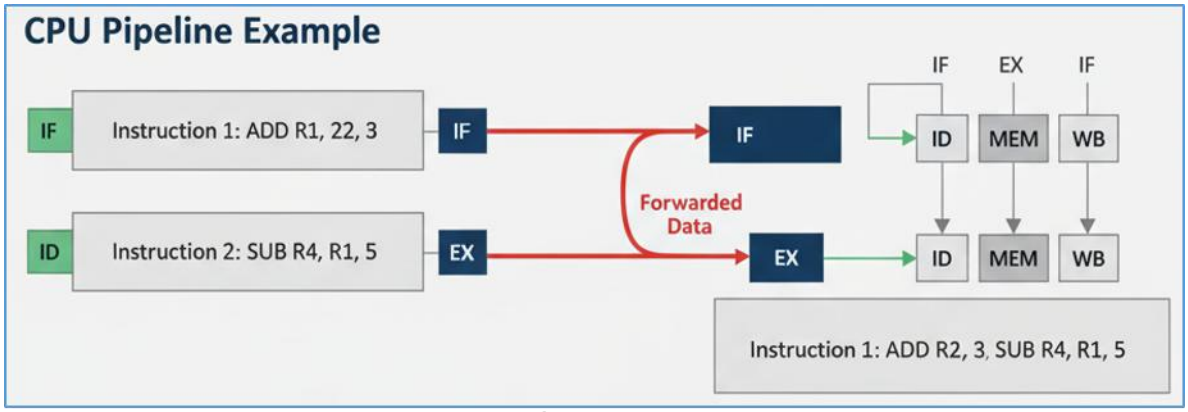
ตัวอย่าง:

คำสั่ง 1: ADD R1, R2, R3 (คำนวณและจะเขียนผลลัพธ์ใส่ R1)

คำสั่ง 2: SUB R4, R1, R5 (ต้องการใช้ค่าจาก R1)

เมื่อคำสั่ง 2 มาถึงสเตจ Decode จะรู้ว่าต้องใช้ค่าจาก R1 แต่คำสั่ง 1 ยังประมวลผลไม่เสร็จและยังไม่ได้เขียนค่ากลับเข้าไปใน R1 ดังนั้นไปป์ไลน์จะ **หยุด (Stall)** คำสั่ง 2 และแทรก Bubble เข้าไปในสเตจ Execute ทำให้คำสั่ง 2 ต้องรออย่างน้อย 2 รอบสัญญาณนาฬิกา จนกว่าคำสั่ง 1 จะเขียนค่ากลับเข้าไปใน R1 เสร็จ

7.8 เทคนิคการแก้ปัญหา Hazard แบบการส่งต่อข้อมูล (Forwarding) หรือ Bypassing

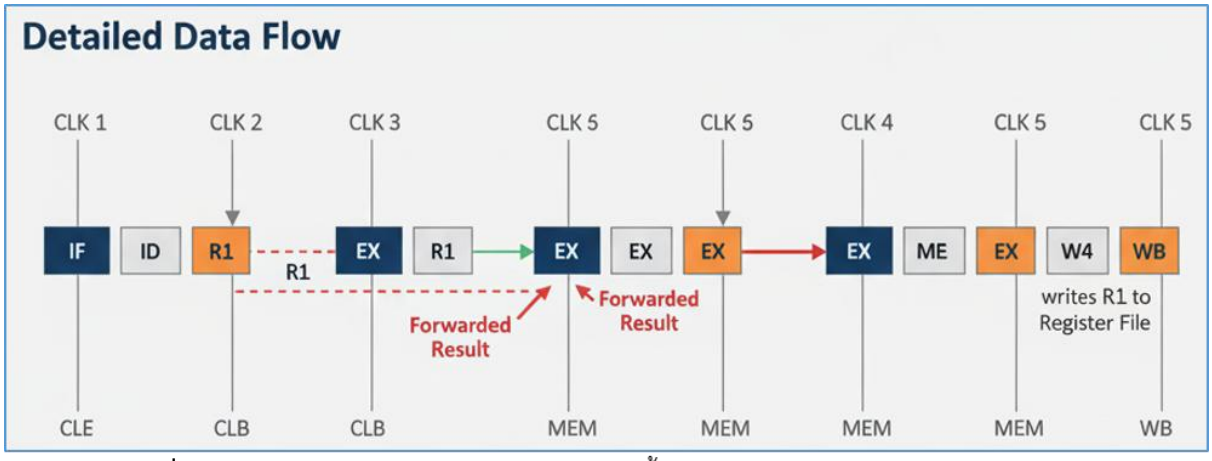


รูปที่ 8 – 26 แผนภาพจำลองการส่งข้อมูลลัดวงจร (Data Forwarding/Bypassing) เพื่อแก้ไขปัญหา Data Hazard

Forwarding คือการแก้ปัญหา **Data Hazard** โดยการ **ส่งผลลัพธ์จากสเตจที่ประมวลผลเสร็จแล้วไปให้สเตจที่กำลังรอใช้ทันที** โดยไม่ต้องรอให้เขียนกลับไปที่รีจิสเตอร์ก่อน เทคนิคนี้ช่วยหลีกเลี่ยงการหยุดรอของไปป์ไลน์ ทำให้ระบบมีประสิทธิภาพสูงขึ้นมาก

ตัวอย่าง:

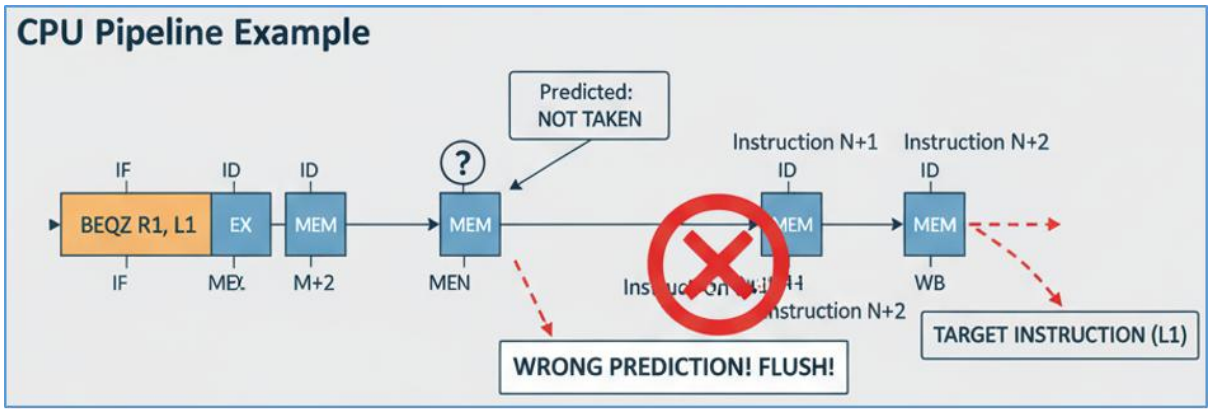
- **คำสั่ง 1:** ADD R1, R2, R3
- **คำสั่ง 2:** SUB R4, R1, R5



รูปที่ 8 – 27 แผนภาพจำลองทิศทางการไหลของข้อมูลโดยละเอียด (Detailed Data Flow) ในกลไก Data Forwarding

แทนที่จะให้คำสั่ง 2 รอจนกว่าคำสั่ง 1 จะเขียนค่ากลับเข้าไปใน R1 (ในสเตจ Write Back) วงจร **Forwarding** จะดักจับผลลัพธ์ที่ได้จากการคำนวณในสเตจ Execute ของคำสั่ง 1 และส่งต่อข้อมูลนั้นไปยังสเตจ Decode ของคำสั่ง 2 โดยตรง ทำให้คำสั่ง 2 สามารถเริ่มต้นประมวลผลได้ทันทีโดยไม่ต้องรอ

7.9 เทคนิคการแก้ปัญหา Hazard แบบการทำนายการทำงาน (Branch Prediction)

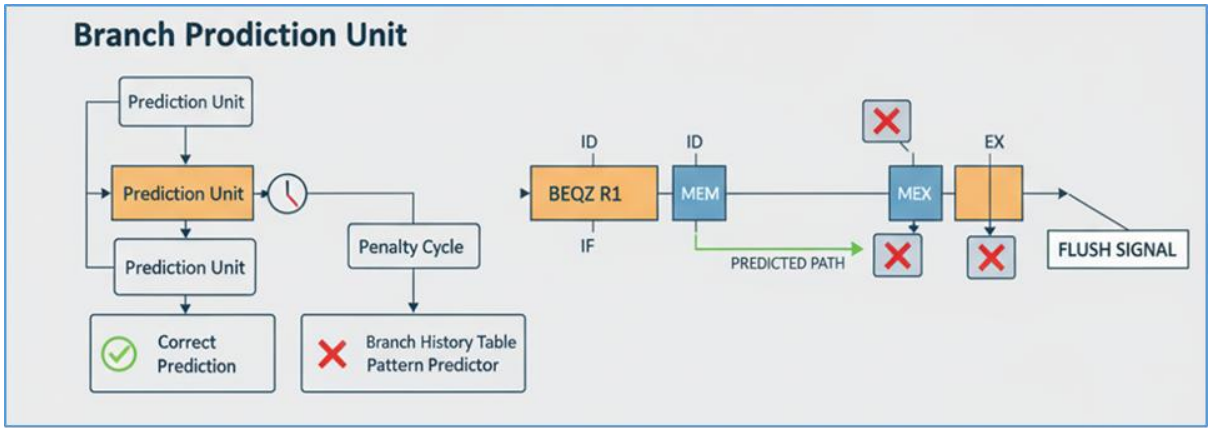


รูปที่ 8 – 28 แผนภาพจำลองสภาวะการล้างระบบไปป์ไลน์ (Pipeline Flush) เมื่อเกิดการทำนายผลสาขาผิดพลาด (Wrong Branch Prediction)

Branch Prediction เป็นเทคนิคที่ใช้แก้ปัญหา **Control Hazard** ที่เกิดจากคำสั่งแบบมีเงื่อนไข เช่น if, jump หรือ goto โดยระบบจะ **คาดเดาว่าเงื่อนไขนั้นจะเป็นจริงหรือเท็จล่วงหน้า** และทำงานตามการคาดเดานั้นไปก่อน

ตัวอย่าง:

คำสั่ง: BEQZ R1, L1 (ถ้า R1 เป็นศูนย์ ให้กระโดดไปที่ L1)

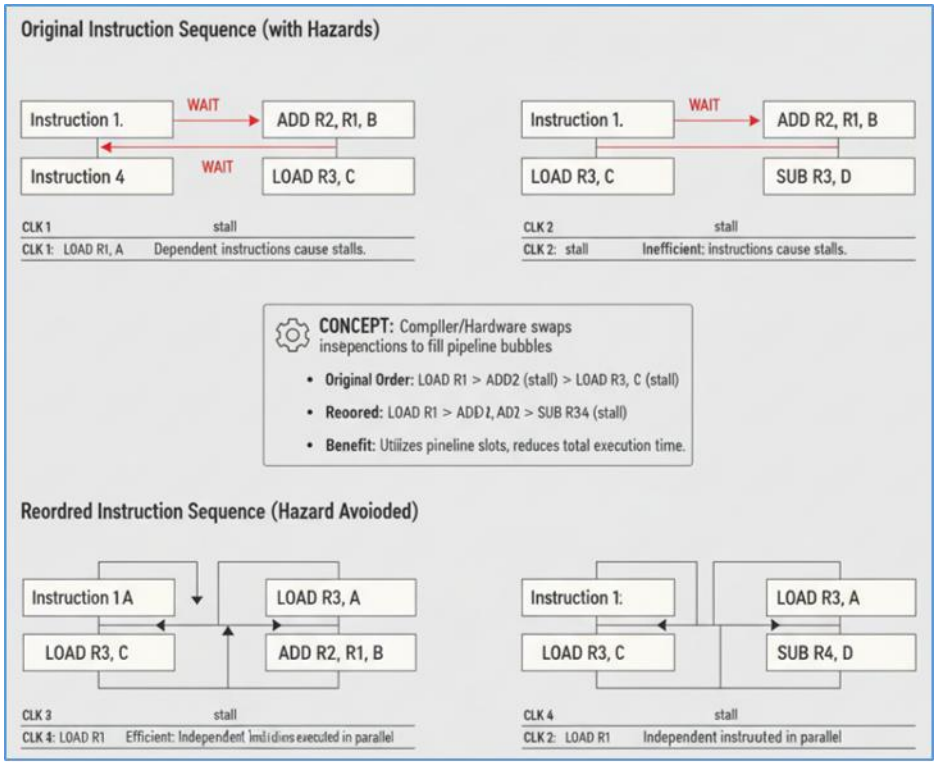


รูปที่ 8 – 29 แผนภาพแสดงโครงสร้างการทำงานของโมดูลทำนายผลสาขา (Branch Prediction Unit) ในสถานการณ์ต่าง ๆ

ถ้าไปป์ไลน์เดาว่าเงื่อนไขจะเป็นเท็จ (จะไม่กระโดด) ก็จะดึงคำสั่งที่อยู่ถัดจาก BEQZ มาประมวลผลล่วงหน้า ถ้าการเดาถูก ก็จะประหยัดเวลาได้อย่างมาก แต่ถ้าการเดาผิด ไปป์ไลน์จะต้อง **ล้าง (Flush)** คำสั่งที่ดึงมาล่วงหน้าทิ้งทั้งหมด และไปดึงคำสั่งที่ถูกต้องมาประมวลผลแทน

7.10 เทคนิคการแก้ปัญหา Hazard แบบการจัดลำดับคำสั่งใหม่ (Instruction Reordering)

Instruction Reordering คือเทคนิคที่คอมไพเลอร์หรือฮาร์ดแวร์จะ **สลับลำดับคำสั่ง** ที่ไม่ขึ้นต่อกันเพื่อหลีกเลี่ยงการเกิด Hazard และช่วยให้ไปป์ไลน์ทำงานได้อย่างราบรื่นขึ้น



รูปที่ 8 – 30 แผนภาพแสดงกลไกการสลับลำดับคำสั่ง (Instruction Reordering) โดยคอมไพเลอร์เพื่อหลีกเลี่ยงสภาวะติดขัด

ตัวอย่าง: ชุดคำสั่งเดิม:

1. LOAD R1, A
2. ADD R2, R1, B (ต้องรอค่าจาก R1)
3. LOAD R3, C
4. SUB R4, R3, D (ต้องรอค่าจาก R3)

จะเห็นว่าคำสั่ง 2 ต้องรอคำสั่ง 1 และคำสั่ง 4 ต้องรอคำสั่ง 3 แต่การประมวลผลของคู่คำสั่ง 1-2 และ 3-4 เป็นอิสระต่อกัน เราสามารถจัดลำดับใหม่ได้ดังนี้:

1. LOAD R1, A
2. LOAD R3, C (ไม่มีการรอเกิดขึ้น)
3. ADD R2, R1, B
4. SUB R4, R3, D

การสลับคำสั่ง 3 มาอยู่ก่อนคำสั่ง 2 ทำให้ไปป์ไลน์สามารถประมวลผลคำสั่ง LOAD R3, C ได้ในขณะที่คำสั่ง ADD R2 กำลังรอค่าจาก R1 ซึ่งเป็นการใช้เวลาให้เกิดประโยชน์สูงสุดและลดการหยุดชะงักของไปป์ไลน์

8. การเปรียบเทียบประสิทธิภาพการทำงานแบบไปป์ไลน์และการทำงานไม่ใช้ไปป์ไลน์ (Compare the performance of Pipelined vs. Non-pipelined operation)

การทำงานแบบไปป์ไลน์ถูกนำมาใช้เพื่อเพิ่มประสิทธิภาพการประมวลผล โดยสามารถวัดได้จาก 2 ค่าหลักคือ **Throughput** และ **Latency**

8.1 Latency (ความหน่วง)

- **คำอธิบาย:** คือ **ระยะเวลาที่ใช้ในการประมวลผลคำสั่งเดียวให้เสร็จสมบูรณ์** ตั้งแต่เริ่มต้นจนสิ้นสุด
- **หน่วย:** มักจะวัดเป็นรอบสัญญาณนาฬิกา (Clock Cycles) หรือวินาที

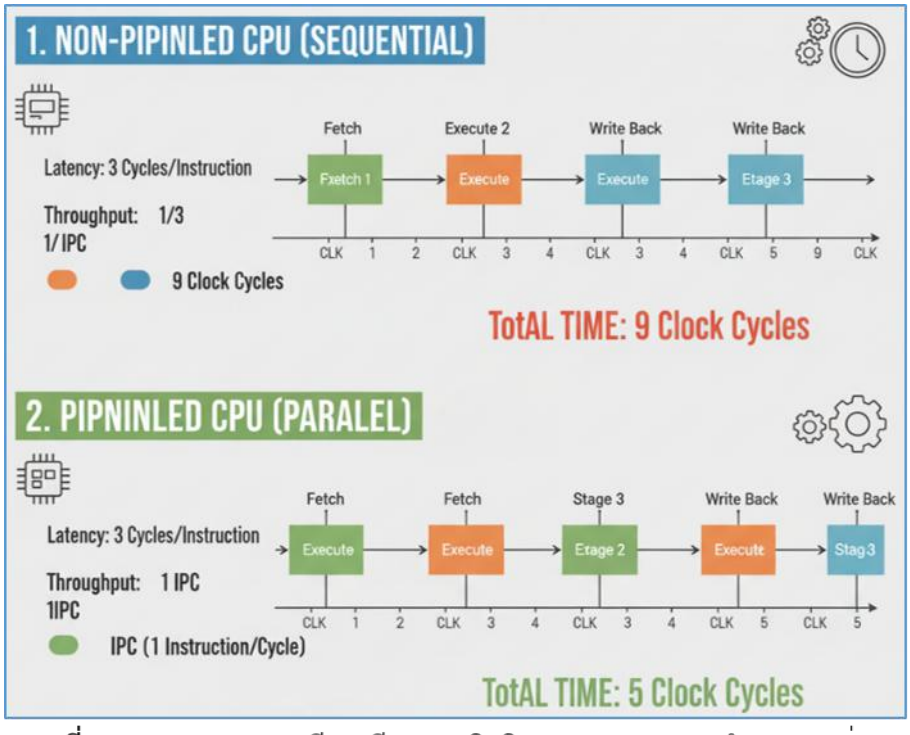
8.2 Throughput (อัตรางาน)

- **คำอธิบาย:** คือ **จำนวนคำสั่งที่ประมวลผลเสร็จในหนึ่งหน่วยเวลา**
- **หน่วย:** มักจะวัดเป็นคำสั่งต่อวินาที (Instructions Per Second - IPS) หรือคำสั่งต่อรอบสัญญาณนาฬิกา (Instructions Per Cycle - IPC)

8.3 การคำนวณและเปรียบเทียบ

สมมติว่าเรามีงาน 3 คำสั่ง แต่ละคำสั่งมี 3 ขั้นตอนย่อย (สเตจ) และแต่ละขั้นตอนใช้เวลา 1 รอบสัญญาณนาฬิกา

แบบที่ 1: ไม่ใช้ไปป์ไลน์ (Non-Pipelined)



รูปที่ 8 – 31 แผนภาพเปรียบเทียบประสิทธิภาพและคาบเวลาทำงานระหว่าง CPU ระบบธรรมดา (Sequential) และระบบไปป์ไลน์ (Pipeline)

- **หลักการ:** ทำงานทีละคำสั่งจนเสร็จสมบูรณ์
- **การคำนวณ Latency:** 3 สเตจ x 1 รอบ/สเตจ = **3 รอบสัญญาณนาฬิกา/คำสั่ง**

- **การคำนวณ Throughput:** ทำงานเสร็จ 1 คำสั่งใน 3 รอบ = 1/3 หรือประมาณ **0.33 IPC**
- **เวลาทั้งหมด:** 3 คำสั่ง x 3 รอบ/คำสั่ง = **9 รอบสัญญาณนาฬิกา**

แบบที่ 2: ใช้ไปป์ไลน์ (Pipelined)

- **หลักการ:** แบ่งงานเป็นสเตจและทำงานพร้อมกัน
- **การคำนวณ Latency:** ยังคงเป็น **3 รอบสัญญาณนาฬิกา/คำสั่ง** (คำสั่งแรกยังต้องผ่านทุกสเตจ)
- **การคำนวณ Throughput:** สามารถปล่อยงานใหม่ได้ทุก 1 รอบสัญญาณนาฬิกา = **1 IPC**
- **เวลาทั้งหมด:** (3 สเตจ + 3 คำสั่ง - 1) รอบ = **5 รอบสัญญาณนาฬิกา**

สรุปผลการเปรียบเทียบ

คุณสมบัติ	ไม่ใช้ไปป์ไลน์	ใช้ไปป์ไลน์
Latency (ความหน่วง)	เท่ากัน	เท่ากัน
Throughput (อัตรางาน)	ต่ำ (0.33 IPC)	**สูง (1 IPC)**
เวลาที่ใช้ทั้งหมด	**สูง (9 รอบ)**	**ต่ำ (5 รอบ)**

ตารางที่ 8-5 การเปรียบเทียบประสิทธิภาพระหว่าง CPU แบบไม่ใช้ไปป์ไลน์และแบบใช้ไปป์ไลน์

จะเห็นว่าแม้ **Latency** ของแต่ละคำสั่งจะเท่ากัน แต่ **Throughput** ของระบบไปป์ไลน์สูงกว่าอย่างเห็นได้ชัด ทำให้โดยรวมแล้วใช้เวลาในการทำงานทั้งหมดน้อยกว่ามาก ซึ่งนี่คือหัวใจสำคัญที่ทำให้ไปป์ไลน์มีประสิทธิภาพเหนือกว่าระบบที่ไม่ใช้ไปป์ไลน์

ชุดข้อสอบเพื่อทบทวนความเข้าใจบทที่ 8 แบบ 4 ตัวเลือก (จำนวน 13 ข้อ)

ข้อ 1 ข้อใดคือข้อดีหลักที่ทำให้สถาปัตยกรรมแบบ ไปป์ไลน์ ถูกนำมาใช้ใน CPU สมัยใหม่?

ก. ลดเวลาในการประมวลผลคำสั่งเดียว (Latency)

ข. ทำให้วงจรควบคุม (Control Unit) ง่ายขึ้น

ค. ช่วยเพิ่มปริมาณงานที่ทำได้ในหนึ่งหน่วยเวลา (Throughput)

ง. ลดการใช้พลังงานของหน่วยประมวลผล

เฉลย: ค.

ข้อ 2 ลำดับขั้นตอนการทำงานของไปป์ไลน์ (Stage) แบบมาตรฐาน 5 ขั้นตอนของสถาปัตยกรรม RISC ทั่วไป ข้อใดเรียงลำดับได้อย่างถูกต้อง?

ก. IF, EX, ID, MEM, WB

ข. IF, ID, EX, MEM, WB

ค. ID, IF, EX, WB, MEM

ง. IF, ID, MEM, EX, WB

เฉลย: ข.

ข้อ 3 ข้อใดคือลักษณะสำคัญของการทำงานแบบ Linear Pipeline?

ก. การไหลของข้อมูลในไปป์ไลน์มีทิศทางเดียว ไม่มีการกระโดดหรือวนกลับ

ข. อนุญาตให้ Stage ต่าง ๆ ทำงานได้หลายฟังก์ชัน

ค. ต้องมีการใช้ตัวควบคุมการทำงานที่ซับซ้อน (Reservation Table)

ง. แต่ละ Stage สามารถประมวลผลคำสั่งเดิมซ้ำได้หลายครั้ง

เฉลย: ก.

ข้อ 4 องค์ประกอบใดที่จำเป็นที่สุดสำหรับ จัดเก็บผลลัพธ์ชั่วคราว และแบ่งแยกการทำงานระหว่างขั้นตอน (Stage) ต่างๆ ของไปป์ไลน์?

ก. Program Counter (PC)

ข. Arithmetic Logic Unit (ALU)

ค. Pipeline Registers (Inter-stage Buffers)

ง. Instruction Register (IR)

เฉลย: ค.

ข้อ 5 เป้าหมายหลักของการ เพิ่มประสิทธิภาพของไปป์ไลน์ คือการดำเนินการในข้อใด?

ก. เพิ่มจำนวน Stage ในไปป์ไลน์ให้มากที่สุด

ข. ลดจำนวนรอบสัญญาณนาฬิกาที่สูญเสียไปกับการรอ (Stall Cycles)

ค. ทำให้เวลาในการเข้าถึงหน่วยความจำ (Memory Access Time) สั้นลง

ง. ใช้หน่วยความจำหลัก (Main Memory) แทน Cache Memory

เฉลย: ข.

ข้อ 6 Data Hazard แบบ RAW (Read After Write) เกิดขึ้นเมื่อใด?

ก. คำสั่งสองคำสั่งใช้ทรัพยากรฮาร์ดแวร์เดียวกันในเวลาเดียวกัน

ข. คำสั่งกระโดดทำให้ CPU ไม่รู้ว่าคำสั่งถัดไปคืออะไร

ค. คำสั่งพยายามเขียนค่าลงใน Register ที่คำสั่งก่อนหน้ายังไม่ได้อ่าน

ง. คำสั่งต่อมาพยายามอ่านค่าจาก Register ที่คำสั่งก่อนหน้ายังเขียนไม่เสร็จ

เฉลย: ง.

ข้อ 7 สาเหตุหลักที่ทำให้เกิด Structural Hazard คืออะไร?

ก. การคาดการณ์คำสั่งกระโดดผิดพลาด

ข. การพึ่งพาข้อมูลระหว่างคำสั่ง

ค. การเข้าถึงทรัพยากรฮาร์ดแวร์เดียวกันพร้อมกันในรอบสัญญาณนาฬิกาเดียว

ง. คำสั่งมีรูปแบบและความยาวไม่เท่ากัน

เฉลย: ค.

ข้อ 8 Control Hazard มีความเกี่ยวข้องโดยตรงกับคำสั่งประเภทใดในชุดคำสั่งของ CPU?

ก. คำสั่ง Branch (การกระโดดแบบมีเงื่อนไข)

ข. คำสั่ง Arithmetic (เช่น ADD, SUB)

ค. คำสั่ง Load/Store (การเข้าถึงหน่วยความจำ)

ง. คำสั่ง NOP (No-operation)

เฉลย: ก.

ข้อ 9 เทคนิค Stalling (Bubble) ที่ใช้ในการแก้ไข Data Hazard ทำงานอย่างไร?

ก. ทำการส่งผลลัพธ์จาก Stage ก่อนหน้าไปยัง Stage ที่ต้องการทันที

ข. แทรกคำสั่งว่างเปล่า (NOP) เข้าไปในไปป์ไลน์เพื่อถ่วงเวลาคำสั่งถัดไป

ค. เปลี่ยนลำดับการทำงานของคำสั่งในโค้ดต้นฉบับ

ง. หยุดสัญญาณนาฬิกาของทุก Stage ในไปป์ไลน์ทั้งหมด

เฉลย: ข.

ข้อ 10 ข้อใดคือหลักการทำงานของเทคนิค Forwarding (Bypassing) ในการแก้ไข Data Hazard?

ก. ดึงผลลัพธ์ที่สร้างเสร็จแล้วจาก Stage ก่อนหน้าไปให้ Stage ที่ต้องการใช้ทันที

ข. ใช้ได้เฉพาะกับ Control Hazard เท่านั้น

ค. ต้องการการแก้ไขซอฟต์แวร์ในการคอมไพล์ใหม่

ง. ต้องรอให้คำสั่งที่สร้างผลลัพธ์เขียนค่าลง Register File ก่อนเสมอ

เฉลย: ก.

ข้อ 11 การทำนาย Branch (Branch Prediction) เกิดข้อผิดพลาด สิ่งที่ CPU ต้องดำเนินการเพื่อแก้ไขคืออะไร?

ก. ทำการ Stalling เพื่อรอผลลัพธ์ของ Branch

ข. ใช้เทคนิค Forwarding ในการส่งค่าที่ถูกต้อง

ค. ทำการ Reorder คำสั่งที่ถูก Fetch เข้ามาใหม่

ง. ล้างคำสั่งที่ถูกดึงเข้ามาล่วงหน้าแล้ว (Flush the Pipeline) และเริ่ม Fetch คำสั่งที่ถูกต้อง

เฉลย: ง.

ข้อ 12 เทคนิค Instruction Reordering ที่ดำเนินการโดย Compiler ช่วยแก้ปัญหา Data Hazard ได้อย่างไร?

ก. คาดการณ์ผลลัพธ์ของคำสั่งก่อนที่มันจะเสร็จสิ้น

ข. แทรก Pipeline Register เพิ่มเติมระหว่าง Stage

ค. จัดลำดับคำสั่งที่ไม่ขึ้นต่อกันมาทำงานก่อนเพื่อสร้างช่องว่างให้คำสั่งที่ต้องรอ

ง. เปลี่ยนคำสั่ง Load/Store ให้เป็นคำสั่ง Register-to-Register

เฉลย: ค.

ข้อ 13 ข้อใดคือตัวชี้วัดประสิทธิภาพที่ Pipelined Architecture เหนือกว่า Non-Pipelined Architecture อย่างชัดเจน?

ก. เวลาที่ใช้ในการประมวลผลคำสั่งเดียว (Instruction Latency)

ข. ปริมาณงานที่ทำได้ในหนึ่งหน่วยเวลา (Throughput)

ค. ความเร็วของสัญญาณนาฬิกา (Clock Speed)

ง. ความซับซ้อนของชุดคำสั่ง (Instruction Set Complexity)

เฉลย: ข.

<u>ส่วนที่ 2: ข้อสอบแบบอธิบาย (Descriptive/Essay Questions) จำนวน 4 ข้อ การออกข้อสอบเป็นแบบการบูรณาการวัตถุประสงค์เข้าด้วยกัน โดยมีคะแนนข้อละ 15 คะแนน</u>

<u>คำถามข้อที่ 1 (15 คะแนน)</u>

คำถาม: จงอธิบายความแตกต่างในหลักการทำงานระหว่างสถาปัตยกรรม CPU แบบ **ไม่ใช้ไปป์ไลน์ (Non-Pipelined)** และแบบ **ไปป์ไลน์ (Pipelined)** โดยใช้แนวคิดของ **Instruction Latency** และ **Throughput** ในการเปรียบเทียบประสิทธิภาพของการทำงานทั้งสองแบบ และสุดท้าย จงสรุปว่าเป้าหมายหลักของการ **เพิ่มประสิทธิภาพของไปป์ไลน์ (Pipeline Optimization)** คือการจัดการกับสิ่งใด

เฉลยและแนวทางการให้คะแนน:

1. **หลักการทำงาน (3 คะแนน):**
    - *Non-Pipelined:* ทำงานแบบเสร็จทีละคำสั่ง โดยแต่ละคำสั่งใช้เวลาเท่ากับผลรวมของทุกขั้นตอน (Sequential)
    - *Pipelined:* ทำงานแบบคาบเกี่ยว (Overlapped) โดยแบ่งคำสั่งเป็น Stage ย่อยๆ และทำงานพร้อมกันหลายคำสั่ง
2. **เปรียบเทียบ Latency (4 คะแนน):** Latency ของ Pipelined ไม่ได้ลดลง (อาจเพิ่มเล็กน้อย) เพราะแต่ละคำสั่งยังคงต้องผ่านทุก Stage ในขณะที่ Non-Pipelined คือผลรวมของเวลาทุก Stage
3. **เปรียบเทียบ Throughput (4 คะแนน):** Throughput ของ Pipelined สูงกว่ามาก เพราะสามารถเริ่มคำสั่งใหม่ได้ทุกรอบสัญญาณนาฬิกา ทำให้จำนวนคำสั่งที่เสร็จสิ้นต่อหน่วยเวลาเพิ่มขึ้น
4. **เป้าหมาย Optimization (4 คะแนน):** เป้าหมายคือการ **ลดหรือขจัด Pipeline Hazards** เพื่อลดจำนวนรอบสัญญาณนาฬิกาที่สูญเสียไปกับการรอ (**Stall Cycles**) อันเป็นผลมาจากการหยุดชะงัก

<u>คำถามข้อที่ 2 (15 คะแนน)</u>

คำถาม: หากคำสั่ง **I2** ต้องการ **อ่าน** ค่าจาก Register ที่คำสั่ง **I1** กำลังจะ **เขียน** ค่าลงไป จงอธิบายว่า **Data Hazard** ประเภทใดที่เกิดขึ้น พร้อมทั้งเปรียบเทียบการแก้ปัญหาโดยใช้เทคนิค **Stalling (Bubble)** กับเทคนิค **Forwarding (Bypassing)** ว่าแต่ละวิธีทำงานอย่างไร และมีข้อดีข้อเสียแตกต่างกันอย่างไร

- **เฉลยและแนวทางการให้คะแนน:**
    1. **ชนิดของ Hazard (3 คะแนน):** คือ **Data Hazard** แบบ **RAW (Read After Write)** เนื่องจาก I2 อ่านก่อน I1 เขียนเสร็จ

2. **กลไก Stalling (4 คะแนน):** คือการ **แทรกคำสั่ง NOP** เข้าไปในไปป์ไลน์ เพื่อ **หน่วงเวลา** (Delay) การทำงานของ I2 ให้ I1 มีเวลาพอที่จะเขียนผลลัพธ์ลง Register File ก่อน
3. **กลไก Forwarding (4 คะแนน):** คือการ **ส่งต่อผลลัพธ์** ที่สร้างเสร็จแล้ว (จาก ALU ของ I1) ไปยัง Stage ที่ต้องการใช้ข้อมูล (EX ของ I2) **ทันที** โดยไม่ต้องรอการเขียนกลับลง Register File
4. **ข้อดีข้อเสียเปรียบเทียบ (4 คะแนน):**
    - *Stalling:* ข้อดี: ใช้งานง่าย, ข้อเสีย: สูญเสียรอบสัญญาณนาฬิกา (ประสิทธิภาพลดลง)
    - *Forwarding:* ข้อดี: ประสิทธิภาพสูง (ไม่เสีย Stall Cycle), ข้อเสีย: ต้องเพิ่มวงจรฮาร์ดแวร์ที่ซับซ้อน
    -

<u>คำถามข้อที่ 3 (15 คะแนน)</u>

คำถาม: จงอธิบายสาเหตุหลักและผลกระทบของ **Structural Hazard** และ **Control Hazard** ในไปป์ไลน์ จากนั้น จงยกตัวอย่าง **แนวทางการแก้ไขพื้นฐาน** ของ Hazard แต่ละชนิดที่ท่านกล่าวถึง

- **เฉลยและแนวทางการให้คะแนน:**
    1. **Structural Hazard (สาเหตุ) (3 คะแนน):** สาเหตุ: คำสั่ง 2 คำสั่งขึ้นไปต้องการใช้ **ทรัพยากรฮาร์ดแวร์เดียวกัน** (เช่น Memory Port) ใน **รอบสัญญาณนาฬิกาเดียวกัน**
    2. **Structural Hazard (การแก้ไข) (4 คะแนน):** การแก้ไข: **การทำสำเนาทรัพยากร** (Resource Replication) เช่น การแยกหน่วยความจำออกเป็น Instruction Memory และ Data Memory (Harvard Architecture)
    3. **Control Hazard (สาเหตุ) (3 คะแนน):** สาเหตุ: เกิดจากคำสั่ง **Branch/Jump** ทำให้ไม่สามารถรู้ได้ว่าคำสั่งถัดไปที่ควรจะ Fetch เข้ามาคือคำสั่งใด จนกว่าเงื่อนไขจะถูกตัดสิน
    4. **Control Hazard (การแก้ไข) (5 คะแนน):** การแก้ไข: **Branch Prediction** (การคาดเดาผลลัพธ์) เพื่อ Fetch คำสั่งล่วงหน้า หรือ **Delayed Branch** (การจัดเรียงคำสั่งที่มีประโยชน์ใน Slot หลัง Branch)

<u>คำถามข้อที่ 4 (15 คะแนน)</u>

- **คำถาม:** องค์ประกอบใดในโครงสร้างไปป์ไลน์ที่สำคัญต่อการถ่ายโอนและจัดเก็บข้อมูลระหว่าง Stage ต่างๆ? และในการประมวลผล **ลูป (Loop)** ของโปรแกรม คำสั่ง Branch ที่อยู่ท้ายลูปจะสร้างปัญหา **Control Hazard** อย่างไร? สุดท้าย จงอธิบายว่าเทคนิค **Branch Prediction** และ **Instruction Reordering** สามารถนำมาใช้บรรเทาปัญหานี้ได้อย่างไร

- **เฉลยและแนวทางการให้คะแนน:**
    1. **องค์ประกอบโครงสร้าง (3 คะแนน):** คือ **Pipeline Registers** หรือ **Inter-stage Buffers** ทำหน้าที่กั้นและเก็บผลลัพธ์ของ Stage ก่อนหน้าเพื่อส่งต่อไป Stage ถัดไป
    2. **Control Hazard ใน Loop (4 คะแนน):** Branch ที่ท้ายลูปทำให้เกิด Control Hazard **ซ้ำๆ** ในเกือบทุกรอบของลูป เมื่อ Branch กระโดดกลับไปต้นลูป คำสั่งที่ Fetch มาผิดจะต้องถูก **Flush** ทำให้เกิดการสูญเสียรอบสัญญาณนาฬิกาอย่างต่อเนื่อง
    3. **การใช้ Branch Prediction (4 คะแนน):** Branch Prediction จะ **เรียนรู้/คาดเดา** ว่า Branch นี้ **จะกระโดด** (Taken) ทำให้ CPU Fetch คำสั่งแรกของลูปเข้ามาล่วงหน้าอย่างถูกต้องในรอบถัดๆ ไป
    4. **การใช้ Instruction Reordering (4 คะแนน):** Compiler จะ **จัดเรียงคำสั่ง** ที่ไม่ขึ้นต่อกันในลูป ให้มาทำงานในตำแหน่งที่ปกติจะเกิด Stall Cycles เพื่อใช้ประโยชน์จากเวลาที่สูญเสียไป